MINISTRY OF EDUCATION AND TRAINING
CAN THO UNIVERSITY
COLLEGE OF INFORMATION AND COMMUNICATION TECHNOLOGY

SOFTWARE ENGINEERING - COHORT 49
CT239H - M01

PROJECT – BASIC TOPIC
SOCIAL POLICY SUPPORT CHATBOT

STUDENT

Cao Tường Hưng B2303873

Can Tho, 08 / 2026

MINISTRY OF EDUCATION AND TRAINING
CAN THO UNIVERSITY
COLLEGE OF INFORMATION AND COMMUNICATION TECHNOLOGY

SOFTWARE ENGINEERING - COHORT 49
CT239H - M01

PROJECT – BASIC TOPIC
SOCIAL POLICY SUPPORT CHATBOT

SUPERVISOR

Ph.D. Phan Phương Lan

STUDENT

Cao Tường Hưng B2303873

Can Tho, 08 / 2026

ABSTRACT

This project develops a web-based chatbot to support Can Tho University students in accessing information about tuition fees, scholarships, tuition exemption and reduction, social support, and student loans. The main objective is to provide accurate and source-grounded answers from collected policy documents.

The system applies a Retrieval Augmented Generation approach. Policy documents are converted from PDF to Markdown, divided into parent and child chunks, embedded using a Vietnamese Bi-Encoder, and indexed in Qdrant. Relevant passages are retrieved, reranked using BGE Reranker v2-m3, and provided to Gemini for answer generation. Structured tuition data and deterministic calculation tools are also used for questions requiring exact financial information.

The chatbot was evaluated using 100 Vietnamese questions across four policy domains. Using an LLM-as-a-judge evaluation method, the system passed 95 out of 100 test cases, achieving 95% accuracy and an average score of 94.65%. The results indicate that the proposed system can provide reliable and efficient support for student financial policy information.

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CTU	Can Tho University
ERD	Entity-Relationship Diagram
FR	Functional Requirement
HNSW	Hierarchical Navigable Small World
HTTP	Hypertext Transfer Protocol
JWT	JSON Web Token
LLM	Large Language Model
NFR	Nonfunctional Requirement
OCR	Optical Character Recognition
RAG	Retrieval-Augmented Generation
SRS	Software Requirement Specification

CHAPTER 1: INTRODUCTION

1.1 PROBLEM STATEMENT

Student financial policy information at a large university is distributed across regulations, annual tuition schedules, scholarship announcements, administrative forms, and external government decisions. A student who asks a practical question rarely knows which document contains the answer or whether a figure represents the actual tuition rate, a statutory ceiling, or the amount used as the basis for exemption and reduction. Documents also differ in publication date, structure, terminology, and target audience. The resulting search task is not merely a keyword lookup: it requires identifying the correct policy domain, academic year, programme, cohort, subject, and eligibility context before extracting an answer.

Conventional website search and manual browsing impose a high cognitive cost. Students must formulate official terminology, open several PDF files, compare tables, and interpret cross-references. Short natural-language questions such as “How much do I pay after a seventy-percent reduction?” combine several facts that may be stored separately. A generic language model can produce fluent explanations but may mix actual tuition with exemption-basis rates, inherit an entity from an unrelated previous answer, or invent a value when the corpus is incomplete. In a student-finance setting, a confident but unsupported number can directly affect planning and trust.

The project therefore treats retrieval correctness, source separation, and controlled calculation as first class requirements. The chatbot must accept conversational Vietnamese, preserve relevant context without allowing previous generated content to contaminate retrieval, and route questions to the correct information lane. Exact tuition-rate questions should use structured records when possible, while broader policy questions should use semantic retrieval over verified documents. When the evidence is absent, the system must state that it cannot find the information rather than fabricate an answer.

The implementation problem also includes operational concerns. Accounts and full conversation histories require durable storage; recent context requires low-latency access; document ingestion must validate uploads and avoid partial indexes; and

administrative functions must be restricted by role. The solution must remain understandable enough for a student project while demonstrating a complete software-engineering process from requirements and architecture to implementation, testing, and evaluation.

1.2 OBJECTIVE

1.2.1 General objective

The general objective is to design and implement a web-based assistant that provides concise, source-grounded support for Can Tho University student financial policies by combining Vietnamese semantic retrieval, deterministic business rules, controlled calculations, and persistent conversational services.

1.2.2 Specific objectives

- Accept natural Vietnamese questions about tuition, scholarships, tuition exemption and reduction, social support, and student loans.
- Separate policy domains and fee concepts before retrieval so that semantically similar but legally different sources are not mixed.
- Use exact structured lookup for tuition rates that depend on programme, cohort, major, subject, or special rule.
- Retrieve relevant policy passages through a parent-child vector architecture and refine the candidates with cross-encoder reranking.
- Invoke deterministic scholarship and tuition-reduction tools only when the question explicitly requires calculation.
- Maintain short-term conversational context in Redis and durable accounts, sessions, messages, and parent documents in PostgreSQL.
- Permit administrators to upload validated PDF documents and roll back every artifact if ingestion fails.
- Evaluate the chatbot using a reproducible test dataset and report quantitative performance across the supported policy domains.

1.3 RESEARCH METHODOLOGY

During the implementation of the project, three main research methods were applied: literature review, observation, and experimentation.

The literature review focused on the technologies used to develop the chatbot system, including large language models, Retrieval-Augmented Generation, embedding models, reranker models, and vector databases. Documents related to tuition fees, scholarships, tuition exemption and reduction, social support, and student loans were also collected from online and official sources. In addition, artificial intelligence tools were used through question and answer interactions to support concept explanation, search keyword suggestion, document summarization, and comparison of technical solutions. Information generated by AI was used only for reference and research guidance. Important information was verified against original documents or reliable sources before being used.

Observation and analysis focused on how students search for policy information through social media, announcements and existing documents. This process helped identify difficulties such as scattered documents, lengthy content, unfamiliar administrative terminology, and difficulty determining which policy applied to a particular case. The chatbot's operation was also observed during question processing, including requirement identification, document retrieval, context selection, and answer generation. Cases involving incorrect source retrieval, policy-category confusion, incomplete answers, or unavailable supporting evidence were recorded for analysis and system improvement.

The experimental method was applied throughout the design, programming, and testing processes. Some programming activities were conducted with AI assistance, including suggesting implementation approaches, explaining errors, proposing source code, and constructing test cases. AI-generated code was reviewed, modified, and tested before being integrated into the system.

The system was experimentally evaluated using questions related to tuition fees, scholarships, tuition exemption and reduction, social support, and student loans. Some test cases were constructed with AI assistance to generate different question formulations, ambiguous questions, incomplete questions, and boundary cases.

1.4 PLAN

Table 1.1. WBS for Chatbot RAG CTU

(Week 1-2): Research RAG/LLM & Collection PDF Data
1.1 Research RAG & LLM system knowledge
1.2 Search and filter PDF documents
(Week 3-4): Build Data Pipeline & Retrieval
2.1 API-based OCR & Markdown Pre-processing
2.2 Research and select Embedding Model
2.3 Implement Chunking mechanism
2.4 Research Vector DB & Deploy basic Retrieval flow
(Week 5-6): Integrate Gemini LLM & Process Table Structure
3.1 Integrate LLM API
3.2 Optimize Chunking (Separate Text and Table flows)
3.3 Develop system utility tools
(Week 7-8): Integrate Reranking, DB & Intent Router, Improve Input Data
4.1 Improve input data
4.2 Setup PostgreSQL, Redis & Reranking
4.3 Build Intent Router
4.4 Develop Query Rewriter for follow-up questions
(Week 9-10): Create Dataset and Write Report
5.1 Pre-filter Metadata before vector search
5.2 Create testing Dataset
5.3 Apply Gemini Rewriter
5.4 Write Project Report

(Week 11-12): Testing &Model Evaluation
6.1 Test Hit@k and Hit rate for Reranker
6.2 Evaluate accuracy
Reverse: Acceptance
7.1 Bug fixing & System optimization
7.2 Acceptance & Finalize Report

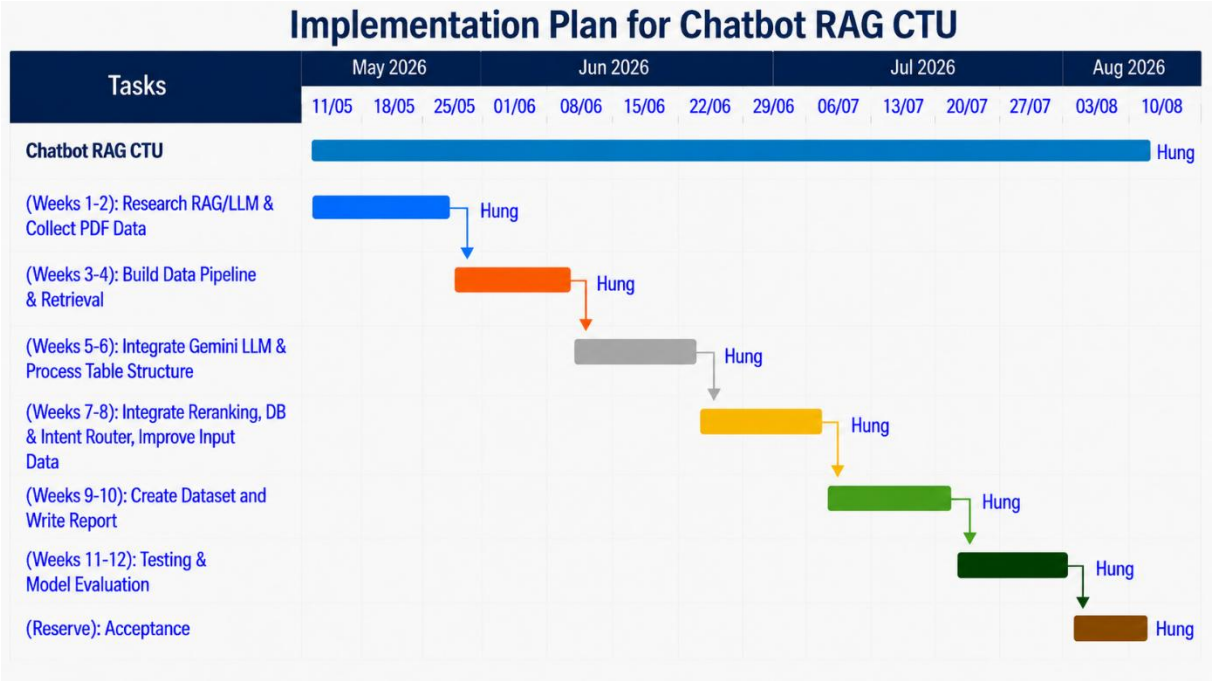

Figure 1.1. Plan for Chatbot RAG CTU

Project progress was monitored against the planned work packages on a weekly basis. Each two-week development phase was defined with a concrete output, such as a prepared document corpus, a working retrieval pipeline, an integrated generation and reranking flow, a testing dataset, or an evaluation result. Completion of these outputs was checked before the project moved to the next dependent phase. This approach reduced the risk of allowing unfinished work from an earlier phase to accumulate near the final deadline.

1.5 PROJECT SCOPE

The information scope is limited to documents collected for CTU student financial support. The active metadata catalog currently classifies sources into tuition, scholarship, social-support, student-loan, and other domains. The application provides registration, cookie-based authentication, conversational question answering, session management, finance tools, and administrator document upload. The retrieval pipeline processes Vietnamese Markdown derived from PDF documents and uses metadata such as domain, content kind, fee kind, academic year, source, status, index version.

CHAPTER 2: THEORETICAL FOUNDATION

2.1 RETRIEVAL-AUGMENTED GENERATION

2.1.1 Retrieval, augmentation, and generation

Retrieval-Augmented Generation combines an information-retrieval system with a generative model. Instead of asking the language model to answer only from parameters learned during pre-training, the system first retrieves passages from a controlled corpus. Those passages are inserted into a prompt together with instructions and the user question. The language model then generates an answer conditioned on explicit evidence. This separation is useful for institutional policy because documents change more frequently than model weights and must remain auditable.

In this project, retrieval is not a single vector query. The system first decides whether a question can be answered by the structured tuition catalog or whether one or more document lanes are required. A lane represents a business distinction such as scholarship, student loan, social support, actual tuition, exemption basis, or exemption policy. Each Qdrant request includes an active-status condition and, when enabled, business metadata filters. The retrieved child vectors lead to larger parent documents in PostgreSQL, and a reranker selects the most relevant parents before context construction.

Augmentation labels every context block with source and business metadata. The labels explicitly distinguish actual tuition from the basis used to calculate exemption or reduction. Headers extracted from Markdown are also added so that a passage retains its position in the original policy. The prompt supplies a routing-specific answer

instruction and requires the model to report missing evidence instead of substituting a semantically similar document. Generation is therefore constrained by both retrieved text and deterministic routing decisions.

2.1.2 Document preparation and metadata

Uploaded PDF documents are validated before processing. The current API accepts safe PDF filenames, checks the media type and PDF signature, enforces a five-mebibyte limit, validates the document class, and requires a consecutive academic-year value where relevant. LlamaParse converts the validated PDF to Markdown. The conversion preserves headings and tables better than plain text extraction, providing structure for downstream splitting.

The metadata catalog assigns each source a domain, content kind, fee kind, academic year, and active status. Technical metadata includes the source filename, effective date, timestamp, SHA-256 checksum, index version, and ingestion-run identifier. Business metadata controls retrieval; technical metadata supports drift detection, reproducibility, and rollback. This distinction prevents operational fields from being confused with the semantics of a student question.

During ingestion, heading-level metadata and business labels are attached before the second parent split, so every derived parent and child retains the same filterable classification. A table-aware splitter keeps Markdown tables intact whenever possible and repeats header rows when a large table must be divided. This behavior is important because tuition schedules often store meaning in column headings that would otherwise be separated from numeric rows.

2.1.3 Parent-child chunking

A single chunk size cannot simultaneously maximize retrieval resolution and generation context. Small chunks are easier to match semantically, but they may omit the surrounding rule or table heading. Large chunks preserve meaning but may contain several unrelated topics. The project applies a small-to-big strategy: child chunks are embedded for high-resolution vector search, while the associated parent passages are returned to the reranker and large language model.

The parent splitter uses a 2,800 character target with 100 character overlap and prefers paragraph, line, sentence, and word boundaries. Child chunks use 400 characters with 50 character overlap. Smart table handling preserves headers in every table fragment. The overlap reduces boundary loss, while metadata labels make the business category visible to both the embedding model and the generated prompt.

2.1.4 Vector search and HNSW

Dense retrieval maps a question and document chunks into the same vector space. Cosine similarity compares their orientation, which approximates semantic relatedness while reducing the influence of vector magnitude. Qdrant stores 768 dimensional child vectors and uses a Hierarchical Navigable Small World index. HNSW builds a multi layer proximity graph that offers approximate nearest-neighbour search with high recall and low query latency. The configured construction parameters balance graph connectivity and indexing cost for the project corpus.

2.1.5 Cross-encoder reranking

A bi-encoder independently represents questions and documents, enabling fast search but losing some token-level interaction. A cross-encoder receives the query and passage together and can evaluate detailed relationships. Running a cross encoder over the entire corpus would be expensive, so the system uses vector search for a broad candidate set and reranks only the retrieved parents. The current pipeline searches for fifteen candidates and retains six after reranking. A soft temporal tie break applies only when relevance scores fall within 0.05, ensuring that a clearly more relevant older policy is not displaced solely by date.

2.1.6 Grounded prompt construction

The final prompt contains the user’s original question, recent human and assistant messages, a retrieval-specific instruction, and labeled evidence. The original wording is preserved for natural response generation even when a clarified search query is used for retrieval. Tool results are appended as structured messages and a final model call verbalizes them. If no authoritative data or correctly filtered passages are available, the API returns an explicit no-result response without invoking speculative generation.

2.2 EMBEDDING MODEL SELECTION

2.2.1 Selected model

The implementation uses bkai-foundation-models/vietnamese-bi-encoder through LangChain's HuggingFaceEmbeddings adapter. The model maps sentences and paragraphs into a 768-dimensional dense vector space and uses a PhoBERT-base-v2 backbone . It is selected for first-stage retrieval because the application corpus and user questions are predominantly Vietnamese and contain formal administrative language.

2.2.1.1 Vietnamese-language specialization

The model is trained for Vietnamese sentence similarity and semantic retrieval. A language-specific encoder is a more direct fit than an English-oriented encoder for questions containing Vietnamese grammar, diacritics, abbreviations, and institutional vocabulary.

2.2.1.2 Policy-domain compatibility

Its training mixture includes Vietnamese-translated MS MARCO and SQuAD together with data from the Legal Text Retrieval Zalo challenge. The latter is relevant to decisions, regulations, eligibility conditions, and administrative passages that resemble the project corpus.

2.2.1.3 Semantic retrieval capability

Dense 768-dimensional representations allow passages to match paraphrased questions even when they do not share exact surface terms. This supports natural student wording that differs from official policy titles.

2.2.1.4 Published model-card evidence

The authors report stronger legal-retrieval results for the fully trained model than for the listed Vietnamese-SBERT and partial-training baselines [1]. These external benchmark figures are evidence about the model, not measurements of this chatbot.

2.2.1.5 Efficient bi-encoder operation

Document chunks are embedded once during ingestion. At runtime only the query is embedded, after which Qdrant performs nearest-neighbour search. This is materially cheaper than joint query-passage scoring over the full corpus.

2.2.1.6 Capacity and storage balance

A 768-dimensional vector offers more representational capacity than common 384-dimensional compact models while remaining practical for the current document collection and local vector store.

2.2.1.7 Technology integration

The model is directly supported by Sentence Transformers and HuggingFaceEmbeddings. Its output dimension matches the configured Qdrant collection and cosine-distance function, so no projection or adapter service is required.

2.2.1.8 Fit with chunking

The model card lists a maximum sequence length of 256 tokens. The 400 character child chunks reduce truncation risk, and the 50 character overlap protects meaning around boundaries.

2.2.1.9 Embedding Model Benchmark Results

The embedding model used in this project is based on PhoBERT-base-v2 and was trained with [1]:

- MS Macro (translated into Vietnamese)
- SQuAD v2 (translated into Vietnamese)
- 80% of the training set from the Legal Text Retrieval Zalo 2021 challenge

Table 2.1. Embedding model benchmark results

Pretrained Model	Training Datasets	Acc@1	Acc@10	Acc@100	Pre@10	MRR@10
Vietnamese-SBERT	-	32.34	52.97	89.84	7.05	45.30
PhoBERT-base-v2	MS MARCO	47.81	77.19	92.34	7.72	58.37
PhoBERT-base-v2	MS MARCO + SQuAD v2.0 + 80% Zalo	73.28	93.59	98.85	9.36	80.73

2.3 RERANKER MODEL SELECTION

The second-stage model is BAAI/bge-reranker-v2-m3, a multilingual cross-encoder derived from BGE-M3. Unlike the embedding model, the reranker does not

create reusable document vectors. It accepts each query-passage pair and returns a relevance score. This joint encoding captures detailed relationships that independent query and document vectors can miss.

The model was selected based on both architectural requirements and project-specific empirical evaluation. The first-stage Vietnamese bi-encoder and Qdrant retrieve up to 15 candidate parent passages to maximize recall. BGE Reranker v2-m3 then reorders these candidates, and only the six highest-ranked parent passages are included in the final prompt. This two-stage design provides a practical balance between retrieval speed, ranking precision, prompt length, and context contamination.

Multilingual support is particularly relevant to the dataset used in this project. Although the policy documents are primarily written in Vietnamese, they also contain English scholarship and programme names, technical terminology, abbreviations, banking terminology, and legal document codes. A multilingual reranker can process these mixed-language expressions without requiring separate ranking models.

A controlled retrieval-only comparison was conducted on ten difficult questions selected from tuition exemption, social support, student loan, and scholarship scenarios. Gemini was excluded from both answer generation and evaluation so that the comparison measured only retrieval and reranking quality. A result was considered a Hit@6 only when the final six parent passages contained both the expected source and the specific fact required to answer the question.

BGE Reranker v2-m3 achieved a Hit@6 of 70% and an MRR of 0.65, while GTE Multilingual Reranker Base achieved a Hit@6 of 60% and an MRR of 0.50. BGE therefore retrieved the required parent passage more frequently and generally placed it at a higher rank.

The end-to-end /chat evaluation provided supporting evidence: the BGE configuration obtained 95% accuracy, compared with 94% for the GTE configuration. However, this one-percentage-point difference was not treated as the primary selection criterion because the end-to-end result may also be affected by Gemini answer generation and LLM-as-a-judge variability. The retrieval-only Hit@6 and MRR measurements therefore provided stronger evidence for model selection.

The system is deployed on a GTX 1650 GPU with 4 GB of VRAM. To remain within this hardware constraint, only the top 15 candidates are reranked and each query passage pair is limited to 512 tokens.

A soft temporal tie-break is applied only when reranker scores differ by no more than 0.05. Semantic relevance therefore remains the primary ranking criterion, while newer documents are preferred only when their relevance scores are nearly equal.

Based on these results, BGE Reranker v2-m3 was selected as the most appropriate practical trade off for the current application. This conclusion is limited to the project's Vietnamese student-finance dataset and available hardware; it does not imply that BGE is universally superior to GTE or other reranking models.

Table 2.2. Comparison of BGE and GTE rerankers

Criterion	BGE Reranker v2-m3	GTE Multilingual Reranker Base
Retrieval-only questions	10	Query and precomputed child vectors
Hit@6	70%	60%
Mean Reciprocal Rank	0.65	0.5
End-to-end accuracy	95%	94%
Main observation	Better fact-bearing Parent ranking	Faster, but lower retrieval quality

Table 2.3. Retrieval and reranking responsibilities

Stage	Model or method	Input	Output	Primary goal
1	Vietnamese Bi-Encoder + Qdrant	Query and precomputed child vectors	Top 15 candidate parents	Fast recall
2	BGE Reranker v2-m3	Query-parent pairs	Top 6 ordered parents	Fine-grained precision
3	Soft temporal tie-break	Near-equal reranker scores	Stable recency-aware order	Current-policy preference without relevance override

2.4 OTHER TECHNOLOGY CHOICES

Table 2.4. Technology stack and rationale

Technology	Project role	Selection rationale
FastAPI	Asynchronous web API	Typed request models, dependency injection, and efficient async integration
PostgreSQL	Persistent relational and parent-document storage	Transactions, relationships, indexing, and JSONB metadata
Redis	Recent conversation context	Low-latency list operations and simple session-scoped keys
Qdrant	Vector and payload store	HNSW search, cosine distance, metadata filters, and collection aliases
Gemini	Answer generation and tool calling	Structured tool-call support and Vietnamese response generation
LlamaParse	PDF-to-Markdown conversion	Free and have limit to 10.000 credits per day.

The architecture deliberately assigns one primary responsibility to each storage system. PostgreSQL is authoritative for users, sessions, full message history, and parent-document content. Redis caches only the recent conversational window. Qdrant stores child vectors and filterable metadata. The structured tuition catalog represents exact monetary facts that should not be diluted by approximate vector similarity. This separation makes failure modes easier to reason about and prevents a single database from being forced into unsuitable workloads.

CHAPTER 3: APPLICATION RESULTS

3.1 SOFTWARE REQUIREMENT SPECIFICATION

3.1.1 Overall description

3.1.1.1 Product perspective

The product is a browser-based client served by a FastAPI application. It is not an unrestricted general-purpose chatbot. Its purpose is to answer CTU student-finance questions from an administered corpus and deterministic business data. The backend

coordinates authentication, conversation state, intent routing, structured lookup, vector retrieval, reranking, tool calling, persistent history, and document ingestion.

3.1.1.2 Actors

Table 3.1. System actors and access boundaries

Actor	Responsibilities	Access boundary
Student user	Register, authenticate, ask questions, continue conversations, and manage owned sessions	Only own sessions and messages
Administrator	Perform all authenticated user actions and upload classified policy PDFs	Upload endpoint additionally requires persisted admin role
External model service	Rewrite constrained follow-ups, parse PDF content, and generate grounded answers	Receives only task-specific data through configured clients

3.1.2 Functional requirements

Table 3.2. Functional requirements

ID	Name	Requirement
FR-01	Account authentication	The system shall register users, verify credentials, issue an HTTP-only JWT cookie, report the current user, and delete the cookie on logout.
FR-02	Question submission	The system shall accept an authenticated query and optional session identifier and return an answer with the active session identifier.
FR-03	Conversational context	The system shall load up to five recent Redis messages scoped by user and session.
FR-04	Safe query rewriting	The system shall rewrite only vague follow-ups and reject rewrites that introduce unsupported entities, numbers, years, or intentions.
FR-05	Intent-aware retrieval	The system shall classify the question and apply domain, content-kind, fee-kind, academic-year, and active-status filters as appropriate.
FR-06	Structured tuition lookup	The system shall answer exact course, programme, cohort, and special-rule tuition questions from authoritative structured records.
FR-07	Controlled calculations	The system shall invoke scholarship or payable-tuition tools only when the required inputs and user intention justify calculation.
FR-08	History management	The system shall list, load, rename, and delete only the current user’s sessions.
FR-09	Administrator upload	The system shall restrict PDF ingestion to authenticated users whose persisted role is admin.

3.1.2.1 FR-01 Account authentication

The system shall register users, verify credentials, issue an HTTP-only JWT cookie, report the current user, and delete the cookie on logout.

3.1.2.2 FR-02 Question submission

The system shall accept an authenticated query and optional session identifier and return an answer with the active session identifier.

3.1.2.3 FR-03 Conversational context

The system shall load recent Redis messages scoped by user and session.

3.1.2.4 FR-04 Safe query rewriting

The system shall rewrite only vague follow ups and reject rewrites that introduce unsupported entities, numbers, years, or intentions.

The original question remains the source of user intent. The rewritten query is accepted only after validation and is used for retrieval, not as a generated answer. This protects clear questions from inheriting unrelated context and prevents the rewriter from injecting a policy domain or academic year.

3.1.2.5 FR-05 Intent aware retrieval

The system shall classify the question and apply domain, content-kind, fee-kind, and active status filters as appropriate.

The route decision may produce more than one lane when the question explicitly combines concepts. Results are deduplicated by document identity. Missing lanes are reported in the answer instruction so that the generator cannot silently substitute another fee type or policy domain.

3.1.2.6 FR-06 - Structured tuition lookup

The system shall answer exact course, programme, cohort, and special-rule tuition questions from authoritative structured records.

3.1.2.7 FR-07 - Controlled calculations

The system shall invoke scholarship or payable tuition tools only when the required inputs and user intention justify calculation.

3.1.2.8 FR-08 - History management

The system shall list, load, rename, and delete only the current user's sessions.

3.1.2.9 FR-09 - Administrator upload

The system shall restrict PDF ingestion to authenticated users whose persisted role is admin.

3.1.2.10 Use Case Diagram

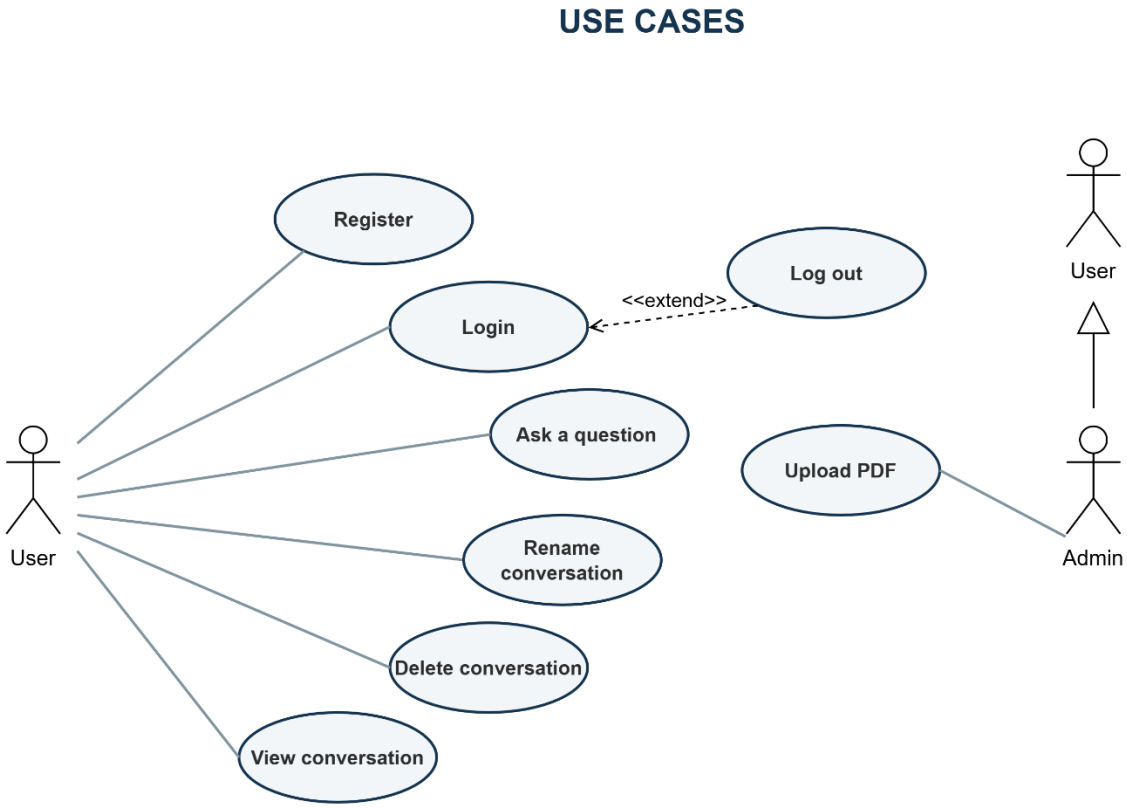

Figure 3.1. Use case diagram

3.1.2.11 Use Case Register

Table 3.3. Use case Register

Use case name: Register	**ID:** UC-01
Primary actor: User	**Priority:** Mandatory
Secondary actor: None	**Classification:** Medium
Brief description: Allows guests to create a new account using a username and password so they can log in, use the chatbot, and manage their conversation history.	

Associated use cases: None
Preconditions: The user is not logged in; the registration page is accessible. **Postconditions:**The new account is stored in the database; the password is hashed using bcrypt; the account is assigned the default role student.
Basic flow of events: 1. The user accesses the chatbot website. 2. The user clicks the Log in button. 3. The system displays the authentication modal. 4. The user selects the Register tab. 5. The system displays the registration form. 6. The user enters a Username and Password. 7. The user clicks the Register button. 8. The interface validates the required fields. 9. The system confirms that the username does not already exist. 10. The system hashes the password using bcrypt. 11. The system creates a new account with the default student role. 12. The system saves the account to the database. 13. The system returns a successful registration response. 14. The interface displays the message: “Registration successful! Please log in.”

Alternative and exception flows: **AF-1:** Switch to the Login Form 1.1. In the authentication modal, the user selects the Login tab. 1.2. The system hides the registration form and displays the login form. **AF-2:** Cancel Register 2.1. The user clicks the button to close the authentication modal. 2.2. The system closes the registration form.
EF-1: Required Information Is Missing 1.1. The user leaves the Username or Password field empty. 1.2. The system asks the user to complete all required fields. 1.3. The flow returns to Step 6 of the basic flow. **EF-2:** Username Already Exists 2.1. The system finds an account with the same username. 2.2. The interface displays: "Username already exists." 2.3. The user enters a different username. 2.4. The flow returns to Step 8 of the basic flow.

3.1.2.12 Use Case Login

Table 3.4. Use case Login

Use case name: Login	**ID:** UC-02
Primary actor: User	**Priority:** Mandatory
Secondary actor: None	**Classification:** Simple
Brief description: This use case allows a registered user to authenticate with a username and password. After successful authentication, the user can access the chatbot and their conversation history.	
Associated use cases: None	
Preconditions: The user has a registered account; user is not currently logged in; the application server and database are available	

Postconditions: On success, a JWT access token is stored in an HTTP-only cookie for seven days, the page is reloaded, and the user's authentication status and conversation sessions are loaded. On failure, no authentication cookie is created and the user remains logged out.

Basic flow of events:

1. The user accesses the chatbot website.
2. The user clicks the Log in button.
3. The user enters a username and password.
4. The user clicks the Log In button.
5. The interface validates that both required fields have been completed.
6. The system searches the database for a user with the provided username.
7. The system creates a JWT access token containing the user ID.
8. The system sets the token in the access_token HTTP-only cookie.
9. The interface closes the authentication modal.
10. The interface reloads the page.
11. The system reads and validates the JWT token from the cookie.
12. The system retrieves the authenticated user from the database.
13. The interface changes the navigation text to "Logout (username)".
14. The interface loads the user's conversation sessions.
15. The user can now use the chatbot and access their conversation history.

Alternative and exception flows:

AF-1: Cancel Login

1.1. The user clicks the close button in the authentication modal.

1.2. The user remains logged out.

EF-1: Required Information Is Missing

1.1. The user leaves the Username or Password field empty.

1.2. The browser detects that a required field has not been completed.

1.3. The browser asks the user to complete the missing field.

1.4. The flow returns to Step 3 of the basic flow.

EF-2: Incorrect Username or Password

2.1. The interface displays “Incorrect username or password.” 2.2. The user remains on the Login form. 2.3. The flow returns to Step 5 of the basic flow. **EF-3:** Server or Database Error 3.1. The server cannot complete the authentication process. 3.2. The interface displays “Login failed.” 3.3. The user remains logged out and may try again later.

3.1.2.13 Use Case Log Out

Table 3.5. Use case Log Out

Use case name: Log out	**ID:** UC-03
Primary actor: User **Secondary actor:** None	**Priority:** Mandatory **Classification:** Simple
Brief description: This use case allows an authenticated user to end the current authenticated session. The system removes the authentication cookie and reloads the chatbot interface.	
Associated use cases: **Extend: Log in**	
Preconditions: The user is logged in; the interface has loaded the current user; the navigation button displays “Logout (username)”. **Postconditions:** The access_token cookie is removed; the page is reloaded; the user is treated as unauthenticated; the navigation button displays “Log in”; protected features require the user to log in again.	
Basic flow of events: 1. The authenticated user clicks “Logout (username)”. 2. The system deletes the access_token cookie. 3. The system returns a successful logout response.	

4. The interface reloads the page. 5. After the page reloads, the interface checks the authentication status through GET /auth/me. 6. The authentication check fails because the access token is no longer available. 7. The interface clears the current user state. 8. The interface displays “Log in” on the navigation button. 9. The user is logged out and must log in again to use protected features.
Alternative and exception flows:
EF-1: Network Connection Error 2.1. The interface cannot connect to the Logout API. 2.2. The request raises a network error. 2.3. The page is not reloaded. 2.4. The authentication cookie may remain available, and the user may still be logged in. 2.5. The user may try to log out again.

3.1.2.14 Use Case Ask A Question

Table 3.6. Use case Ask a Question

Use case name: Ask a question	**ID:** UC-04
Primary actor: User	**Priority:** Mandatory
Secondary actor: None	**Classification:** Complex
Brief description: This use case allows an authenticated user to ask questions about student policies, including scholarships, student loans, social support, tuition fees, and tuition exemption or reduction. The system retrieves relevant information, generates an answer, displays it to the user, and saves the conversation history.	
Associated use cases: None	

Preconditions: The user has logged in successfully; a valid JWT is available in the access_token cookie; the chatbot services have been initialized; Redis, the retrieval engine, and the language model are available; the user is on the chatbot interface.

Postconditions: The generated answer is displayed in the conversation; the user's question and the chatbot's answer are stored; the user can access any session chat.

Basic flow of events:

1. The user enters a policy-related question in the message input field.
2. The user clicks the Send button or presses Enter.
3. The system validates the question and the user's authentication status.
4. The system displays the user's question and a typing indicator.
5. The system creates a new conversation or continues the currently selected conversation.
6. The system retrieves the recent conversation history.
7. The system identifies the user's intent and retrieves relevant information from the document repository.
8. The system uses the retrieved information and conversation history to generate an answer.
9. The system displays the generated answer in the conversation area.
10. The system saves the user's question and the generated answer.
11. The system updates the conversation list.
12. The user reads the answer and may continue asking questions.

Alternative and exception flows:

AF-1: Start a new conversation

1.1 The user has not selected an existing conversation.

1.2. The system generates a new conversation ID.

1.3. The system processes the question and creates a new conversation.

1.4. The conversation title is generated from the first 50 characters of the question.

1.5. The flow continues from Step 6 of the basic flow.

AF-2: Continue an existing conversation

2.1. The user has selected an existing conversation.

2.2. The system uses the current conversation ID.

2.3. The system retrieves the recent messages from that conversation.

2.4. The new question and answer are added to the same conversation.

2.5. The flow continues from Step 6 of the basic flow.

AF-3: A Tool Is Using For Caculation

3.1. The question requires a scholarship or tuition calculation.

3.2. The language model requests the appropriate calculation tool.

3.3. The calculation result is provided to the language model.

3.4. The language model generates the final answer.

3.5. The flow continues from Step 9 of the basic flow.

EF-1: Empty Question

1.1. The question is empty or contains only whitespace.

1.2. The system does not send the question to the server.

1.3. No conversation or message is created.

1.4. The user remains on the chat interface and may enter another question.

EF-2: User Is Not Authenticated

2.1. The authentication cookie of no longer exists.

2.2. The system displays the error in the conversation area.

2.3. The question is not processed or saved.

2.4. The user must log in again.

EF-3: Question Processing Error

3.1. An unexpected error occurs in server.

3.2. The system removes the typing indicator.

3.3. The system displays a message “Xin lỗi, không thể kết nối tới máy chủ. Vui lòng thử lại sau.”

3.4. The user try again later.

3.1.2.15 Use Case Rename Conversation

Table 3.7. Use case Rename Conversation

Use case name: Rename Conversation	**ID:** UC-05
Primary actor: User	**Priority:** Desirable
Secondary actor:	**Classification:** Simple
Brief description: This use case allows an authenticated user to change the title of a conversation that belongs to them.	
Associated use cases: None	
Preconditions: The user is logged in; the conversation list has been loaded; the selected conversation exists and belongs to the authenticated user. **Postconditions:** The trimmed conversation title is stored in PostgreSQL; the conversation list is refreshed and displays the updated title.	
Basic flow of events: 1. The user views the conversation list. 2. The user clicks the Edit button of a conversation. 3. The system displays a prompt containing the current conversation title. 4. The user enters a new title. 5. The user confirms the new title. 6. The interface removes leading and trailing whitespace from the title. 7. The system updates the conversation title and commits the change. 8. The system returns a successful response. 9. The interface reloads the conversation list. 10. The updated title is displayed to the user.	
Alternative and exception flows: **AF-1:** Cancel Renaming 1.1. The user closes or cancels the rename prompt. 1.2. The interface does not send a request to the server. 1.3. The original conversation title remains unchanged.	

AF-2: Empty Conversation Title 2.1. The user leaves the title empty or enters only whitespace. 2.2. The interface detects that the trimmed title is empty and does not send a request to the server. 2.3. The original conversation title remains unchanged.
EF-1: Conversation Is Not Found or Does Not Belong to the User 1.1. The conversation does not exist, or its user ID does not match the authenticated user ID. 1.2. The conversation title is not updated. 1.3. The frontend does not display an error message or refresh the conversation list.

3.1.2.16 Use Case Delete Conversation

Table 3.8. Use case Delete Conversation

Use case name: Delete conversation	**ID:** UC-06
Primary actor: User	**Priority:** Desirable
Secondary actor:	**Classification:** Simple
Brief description: This use case allows an authenticated user to permanently delete one of their conversations and its messages.	
Associated use cases:	
Preconditions: The user is logged in; the conversation list has been loaded; the selected conversation exists and belongs to the authenticated user. **Postconditions:** The conversation and its messages are deleted from PostgreSQL; the corresponding temporary conversation history is deleted from Redis; the conversation list is refreshed.	

Basic flow of events: 1. The user clicks the Delete button of a conversation. 2. The system displays a confirmation message. 3. The user confirms the deletion. 4. The system verifies that the conversation belongs to the authenticated user. 5. The system deletes the conversation and its messages from PostgreSQL. 6. The system deletes the corresponding conversation history from Redis. 7. The system refreshes the conversation list. 8. The deleted conversation is no longer displayed.
Alternative and exception flows: **AF-1:** Cancel Deletion 1.1. The user cancels the confirmation message. 1.2. The system does not send a deletion request. 1.3. The conversation and its messages remain unchanged. **AF-2:** Delete the Currently Selected Conversation 2.1. The deleted conversation is the currently selected conversation. 2.2. The system deselects the current conversation. 2.3. The system removes all displayed messages. 2.4. The system displays the empty chat state. 2.5. The system refreshes the conversation list.
EF-1: Conversation Is Not Found or Does Not Belong to the User 1.1. The conversation does not exist, or it belongs to another user. 1.2. The conversation is not deleted. 1.3. The frontend does not display an error message. 1.4. The conversation list remains unchanged.

3.1.2.17 Use Case View Conversation

Table 3.9. Use case View Conversation

Use case name: View Conversation	**ID:** UC-07

Primary actor: User **Secondary actor:**	**Priority:** Mandatory **Classification:** Simple
Brief description: This use case allows an authenticated user to select a previous conversation and view all of its messages.	
Associated use cases:	
Preconditions: The user is logged in; the conversation exists and belongs to the authenticated user; the conversation list has been loaded. **Postconditions:** The selected conversation is marked as active; its messages are displayed in chronological order.	
Basic flow of events: 1. The user views the conversation list. 2. The user selects a conversation. 3. The system clears the messages currently displayed. 4. The system verifies that the selected conversation belongs to the authenticated user. 5. The system retrieves the conversation messages from PostgreSQL. 6. The system displays the messages in chronological order. 7. The user can read the conversation or continue asking questions.	
Alternative and exception flows: **AF-1:** Select Another Conversation 1.1. The user selects a different conversation. 1.2. The system deselects the current conversation. 1.3. The system removes the currently displayed messages. 1.4. The system loads and displays the messages of the newly selected conversation. 1.5. The flow continues from Step 4 of the basic flow.	
EF-1: User Is Not Authenticated	

1.1. The authentication cookie is missing, invalid, or expired. 1.2. The conversation messages are not returned. 1.3. The frontend does not display an error message. 1.4. The message area remains empty.

3.1.2.18 Use Case Upload PDF

Table 3.10. Use case Upload PDF

Use case name: Upload PDF	**ID:** UC-08
Primary actor: Admin	**Priority:** Mandatory
Secondary actor:	**Classification:** Complex
Brief description: This use case allows an administrator to upload a PDF document, convert it to Markdown, assign document metadata, and ingest it into the RAG knowledge repository.	
Associated use cases: None	
Preconditions: The administrator is logged in with the admin role; the PDF file is available. **Postconditions:** The original PDF is stored in the completed-document directory; the cleaned Markdown file and metadata are created; the document is ingested into the RAG repository; the upload modal is closed and a success message is displayed.	
Basic flow of events: 1. The administrator clicks Upload PDF. 2. The system displays the upload form. 3. The administrator selects a PDF file and its document class. 4. If required, the administrator enters the academic year. 5. The administrator clicks Upload. 6. The system validates the file and document information. 7. The system converts the PDF into Markdown and cleans the generated content.	

8. The system stores the document metadata and ingests the document into the RAG repository.
9. The system moves the original PDF to the completed-document directory.
10. The system closes the upload form and displays a success message.

Alternative and exception flows:

AF-1: Cancel Upload

3.1. The administrator clicks Cancel, the close button, presses Escape, or clicks outside the modal.

3.2. The system closes and resets the upload form.

3.3. No file is uploaded or processed.

AF-2: Document Class Requires an Academic Year

4.1. The administrator selects Actual Tuition Rate or Tuition Exemption Basis.

4.2. The system displays the Academic Year field.

4.3. The administrator enters two consecutive years in YYYY-YYYY format, such as 2025-2026.

4.4. The flow continues from Step 5 of the basic flow.

AF-3: Document Class Does Not Require an Academic Year

4.1. The administrator selects another supported document class.

4.2. The system hides and clears the Academic Year field.

4.3. The flow continues from Step 5 of the basic flow.

EF-1: Invalid Upload Form

1.1. One of the following validation errors occurs:

- No file is selected.
- The file does not have a .pdf extension.
- The MIME type is not application/pdf or application/x-pdf.
- The file is empty.
- The file is larger than 5 MiB.
- The document class is invalid.

- A required academic year is missing or is not in the correct consecutive-year format.

1.2. The system displays the corresponding validation message.

1.3. The invalid field receives focus.

1.4. No request is sent to the server.

1.5. The administrator corrects the information and tries again.

EF-2: Network Connection Error

2.1. The interface cannot connect to the upload API.

2.2. The system displays “Unable to connect to the server. Please try again.”

2.3. The upload controls are enabled again.

2.4. The selected document is not confirmed as uploaded.

2.5. The administrator may try again.

3.1.3 Nonfunctional requirements

Table 3.11. Nonfunctional requirements

ID	Category	Requirement
NFR-01	Security	Passwords shall be bcrypt-hashed; JWTs shall be stored in HTTP-only cookies; admin operations shall verify the persisted role.
NFR-02	Input safety	The upload boundary shall reject traversal names, reserved filenames, invalid media types, forged signatures, and oversized files.
NFR-03	Data integrity	Document metadata mutation and upload rollback shall be atomic and shall not overwrite concurrent additions.
NFR-04	Reliability	The system shall handle service or processing failures without corrupting stored data and shall return an explicit error or no-result response when normal processing cannot be completed.
NFR-05	Performance	The system shall limit vector retrieval to 15 candidates and reranking to 6 candidates to constrain response latency and GPU memory consumption under the project hardware configuration.
NFR-06	Maintainability	Authentication, chat, history, document, routing, catalog, RAG, and metadata responsibilities shall remain modular.
NFR-07	Auditability	Retrieved contexts shall retain source, domain, fee kind, year, index version, and ingestion identifiers.
NFR-08	Usability	The browser client shall expose clear authentication, chat, history, and upload states with actionable error messages.

3.2. SOFTWARE DESIGN

3.2.1. Application architecture

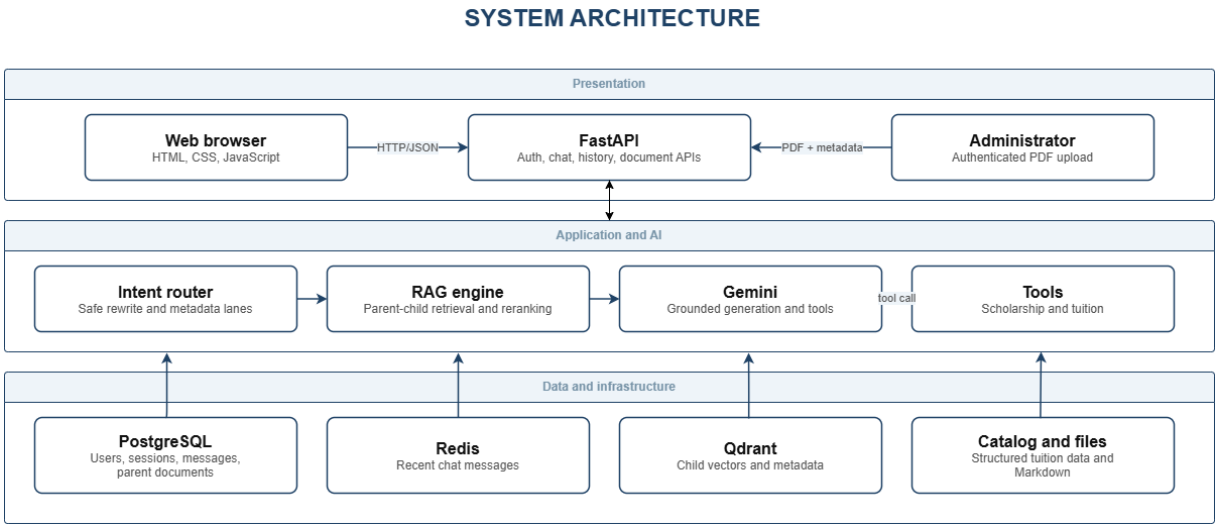

Figure 3.2. Application architecture

The presentation layer consists of static HTML, CSS, and JavaScript served from the same FastAPI origin. The browser calls authentication, session, chat, and document endpoints with cookies. The application layer coordinates typed request models and authorization dependencies. It separates deterministic intent and catalog logic from RAG retrieval and from the generative model.

The AI orchestration layer uses a low-temperature Gemini model only for constrained follow-up rewriting and Gemini for grounded answer generation and tool calling. The RAG engine is responsible for ingestion, vector retrieval, parent restoration, reranking, and metadata filtering. Scholarship and payable-tuition calculations are deterministic functions, ensuring that arithmetic is not delegated to unconstrained text generation.

The data layer separates workloads. Redis stores recent role-content pairs for low-latency context. PostgreSQL stores users, sessions, messages, and parent documents. Qdrant stores child embeddings and metadata used for filtering. The tuition catalog stores exact rate records and rules. The document metadata manifest ensures that every active Markdown source has a validated business classification.

Table 3.12. Architecture-layer responsibilities

Layer	Principal components	Responsibility
Presentation	index.html, support.html, script.js, style.css	User interaction, validation feedback, and API calls
API	auth.py, chat.py, history.py, document.py	Request boundaries, authorization, orchestration, and responses
Domain services	query_intent.py, tuition_catalog.py, document_metadata.py	Deterministic classification and business data
AI services	rag_engine.py, ocr_service.py, Gemini	Ingestion, retrieval, reranking, parsing, and generation
Persistence	PostgreSQL, Redis, Qdrant, JSON/Markdown files	Durable data, recent context, vectors, and source artifacts

3.2.2. Data design

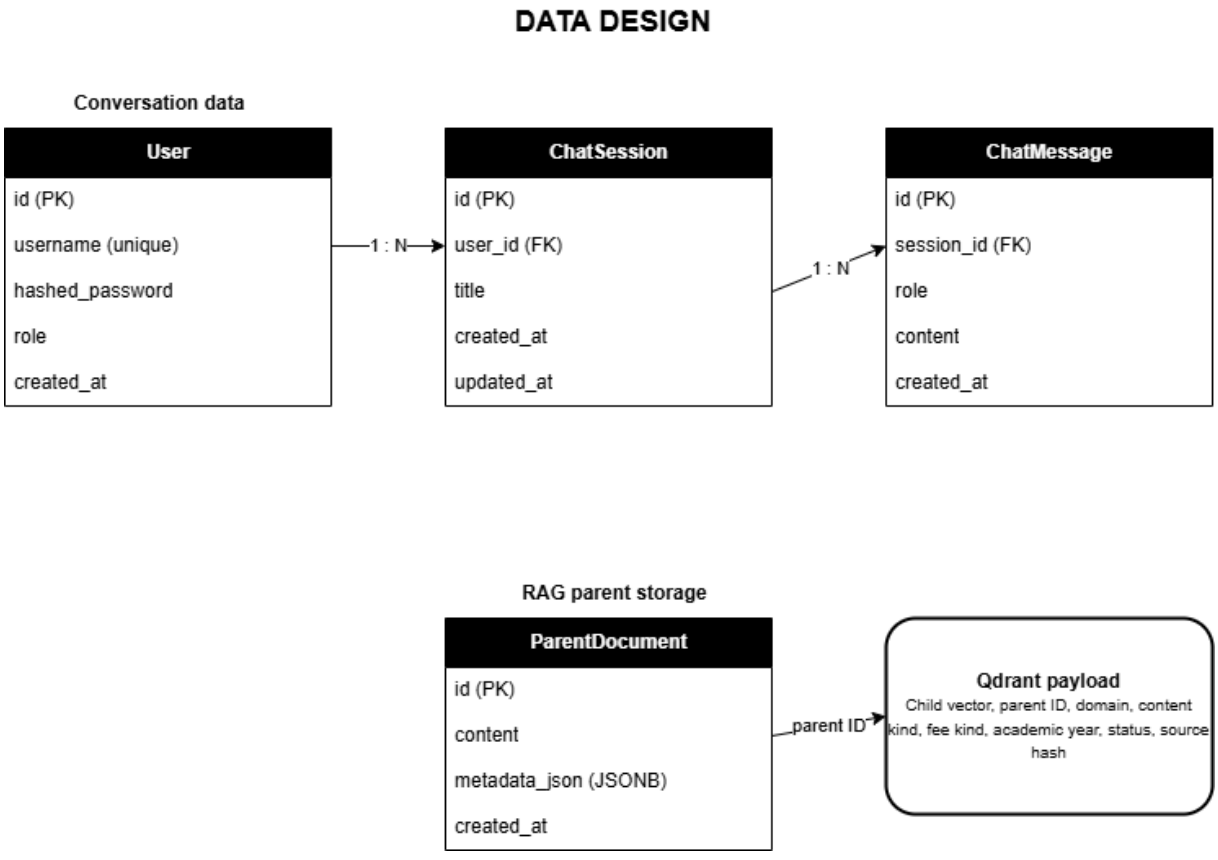

Figure 3.3. Relational and vector data design

The User-to-ChatSession and ChatSession-to-ChatMessage relationships are one-to-many with delete-orphan cascades. UUID version 7 string identifiers provide time-ordered identifiers without a central integer sequence. Message roles distinguish human and AI entries. Session titles are initially derived from the first query and can be renamed. ParentDocument stores the original parent text and JSONB metadata under the identifier referenced by Qdrant child payloads.

Table 3.13. Users table structure

STT	Field name	Data type	Note	Description
1	id	String	Primary key	Unique identifier of the user. The default function generates a version 7 UUID as a string.
2	username	String	UNIQUE; INDEX;NOT NULL	The value must be unique and indexed to support lookup during login.
3	hashed_password	String	NOT NULL	Hashed password; plain text passwords are not stored.
4	role	String	Default: student	Account role. The application uses the admin value to check administrative privileges; new users receive student by default.
5	created_at	DateTime		The time the user record was created, including timezone information.

Table 3.14. Chat session table structure

STT	Field name	Data type	Note	Description
1	id	String	Primary key	Unique identifier of the user. The default function generates a version 7 UUID as a string.
2	user_id	String	FK → users.id; NOT NULL	Links the conversation with the owning account.
3	title	String	NULLABLE	Title of the conversation. May not have a value before the system creates or updates the title.
4	created_at	DateTime		The time the conversation was created.
5	updated_at	DateTime		The most recent update time of the conversation; refreshed when the ORM updates the record.

Table 3.15. Chat message table structure

STT	Field name	Data type	Note	Description

1	id	String	Primary key	Unique identifier of the message, generated as a version 7 UUID.
2	session_id	String	FK→chat_sessions.id ;NOT NULL; INDEX	Links the message to the conversation and supports fast querying of history by session.
3	role	String	NOT NULL	The role of the sender.
4	content	Text	NOT NULL	The full content of the message.
5	created_at	DateTime		The time was created.

Table 3.16. Parent document table structure

STT	Field name	Data type	Note	Description
1	id	String	Primary key	Identifier of the parent document. The value is provided by the RAG pipeline and is used to map with parent_id in the Qdrant payload.
2	content	Text	NOT NULL	The full text content of the parent document used to restore context after vector retrieval.
3	metadata_json	JSONB	NULLABLE	Metadata of the document.
4	created_at	DateTime		The time the parent document was created.

3.2.2.1. Document metadata

Table 3.17. Business and technical metadata

Field	Purpose
domain	Separates tuition, scholarship, loan, support, and other sources
content_kind	Distinguishes policies, rate tables, announcements, forms, and procedures
fee_kind	Separates actual tuition, exemption basis, and not-applicable records
academic_year	Scopes year-specific material
status	Excludes inactive content
index_version	Identifies immutable vector build
ingest_run_version	Scopes rollback

3.2.2.2. Evaluation dataset design

The evaluation dataset contains 100 Vietnamese questions designed to represent common student requests within the information scope of the chatbot. The dataset is balanced across four domains, with 25 questions in each domain. This balanced distribution prevents the overall accuracy from being dominated by a single topic and makes domain-level performance easier to compare.

Table 3.18. Evaluation dataset distribution

Domain	Question IDs	Number of questions
Tuition exemption and learning-cost support	1–25	25
Tuition	26–50	25
Student loans	51–75	25
Scholarships	76–100	25

Each test case contains three core elements: a natural-language student question, an expected answer, and the source Markdown filename from which the expected information was obtained.

3.2.3. Detailed design

3.2.3.1. Document ingestion pipeline

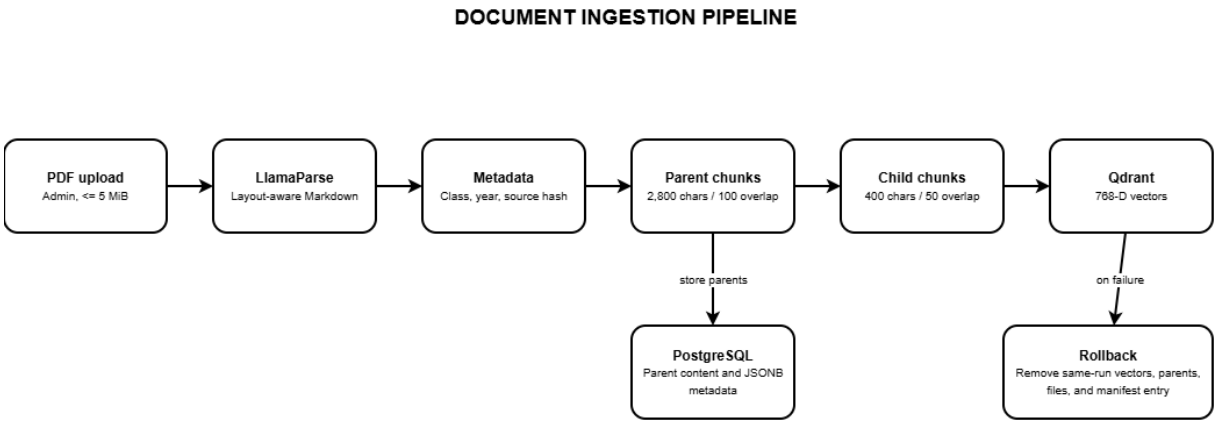

Figure 3.4. Document ingestion and rollback pipeline

1. Validate the upload boundary before accepting a file path or writing untrusted content.
2. Generate a unique ingestion-run identifier and a collision-safe working filename.
3. Convert the PDF to Markdown through LlamaParse and classify the new source through server-owned mappings.
4. Extract effective date and attach business and technical metadata to heading-derived parent blocks.
5. Preserve tables while splitting large prose parents and generate 400-character child chunks.

6. Embed children with the Vietnamese Bi-Encoder, store vectors in Qdrant, and store parents in PostgreSQL.
7. Move final artifacts and atomically update the metadata manifest only after successful indexing.
8. On any partial failure, purge the same ingestion run and remove generated files and catalog entries.

3.2.3.2. Chat query pipeline

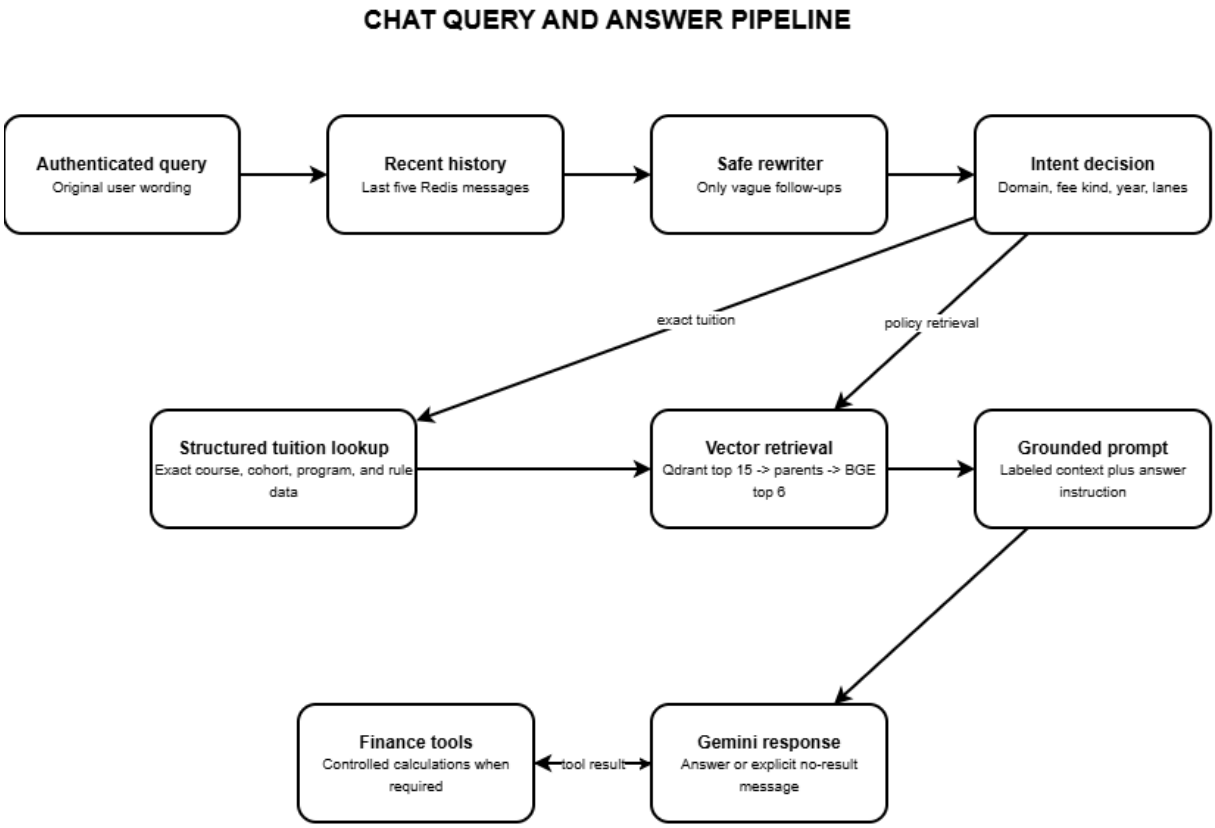

Figure 3.5. Query routing, retrieval, and answer generation

The API creates a new UUID session when the request does not provide one. Redis history is scoped by both user and session, preventing one account or conversation from inheriting another user's context. Only the last five entries are supplied to the answer prompt. When a persisted session is loaded, the history endpoint repopulates Redis with the latest stored messages.

3.2.3.2. Safe follow-up rewriting

A heuristic first determines whether the current question is genuinely referential or elliptical. Clear questions bypass the rewriter. For a vague follow-up, the Gemini-3.1-flash-lite model receives only the previous human question and the current question, not

the previous AI answer. A validator rejects multiline output, answer-like text, unrelated numbers, new years, new entities, or an intent not supported by the two user questions. Rejection and provider failure both fall back to the original query.

3.2.3.3. Intent routing and retrieval lanes

The classifier recognizes actual tuition, exemption basis, exemption policy, calculation, combined tuition concepts, scholarship, student loan, social support, ambiguous tuition, and other topics. It can extract an academic year and identify whether the route came from the original or clarified query. Each decision maps to immutable retrieval-lane objects containing domain and content constraints. Hard domain separation reduces cross-topic contamination, such as matching SCC scholarship content to SCIC or retrieving loan documents for a tuition question.

3.2.3.4. Structured tuition lookup

Exact tuition questions first enter the structured catalog. The catalog normalizes Vietnamese text and extracts programme, cohort, major, subject, education level, study mode, and special-rule phrases. It can request clarification when an essential dimension is absent. A successful exact lookup is authoritative: the API does not add approximate vector passages that could dilute or contradict the monetary result. A rewritten query is used only when the catalog confirms that the rewrite did not inject an exemption intent or other unsafe entity.

3.2.3.5. Vector retrieval and reranking

For policy questions, the RAG engine creates request-scoped Qdrant filters and retrievers. An active-status condition is always applied. Business metadata filtering can be controlled during rollout so a new collection may be activated safely. The vector store returns child matches, the ParentDocumentRetriever restores larger PostgreSQL parents, and the BGE cross-encoder scores query-parent pairs. Duplicate parents are removed before prompt construction.

3.2.3.6. Tool calling and response construction

Gemini receives two deterministic tools. The scholarship tool uses GPA, conduct score, and discipline group to apply a defined award schedule. The tuition tool accepts actual tuition, the applicable exemption-basis ceiling, and the reduction percentage; it

validates the percentage and calculates the payable amount using the correct cap. If the model requests an unknown tool, the server returns an explicit tool error rather than executing arbitrary code.

3.2.3.7. History persistence

After a response is produced, the API appends human and AI messages to Redis and trims the list to the configured window. A background task creates or updates the PostgreSQL session and writes both messages. History endpoints enforce ownership before listing messages, renaming a session, or deleting it. Deleting a session cascades relational messages and also removes the corresponding Redis key.

3.2.3.8. Temporal Soft Tie-Break in Reranking

Temporal Soft Tie Break in Reranking In standard retrieval pipelines, sorting solely by semantic relevance can surface outdated documents, while sorting strictly by time might discard highly relevant historical information. To balance this trade-off, the system implements a soft tie-break bucketing algorithm during the cross encoder reranking phase. Documents are initially ranked by their relevance scores. The algorithm then groups them into buckets if their score differences fall within a strict tolerance threshold . Within each bucket of effectively equally relevant candidates, documents are dynamically reordered by their timestamps in descending order. This mechanism ensures that a clearly more relevant older document is not unfairly displaced solely by its date, while simultaneously guaranteeing that among equally relevant documents, the most up-to-date information is prioritized.

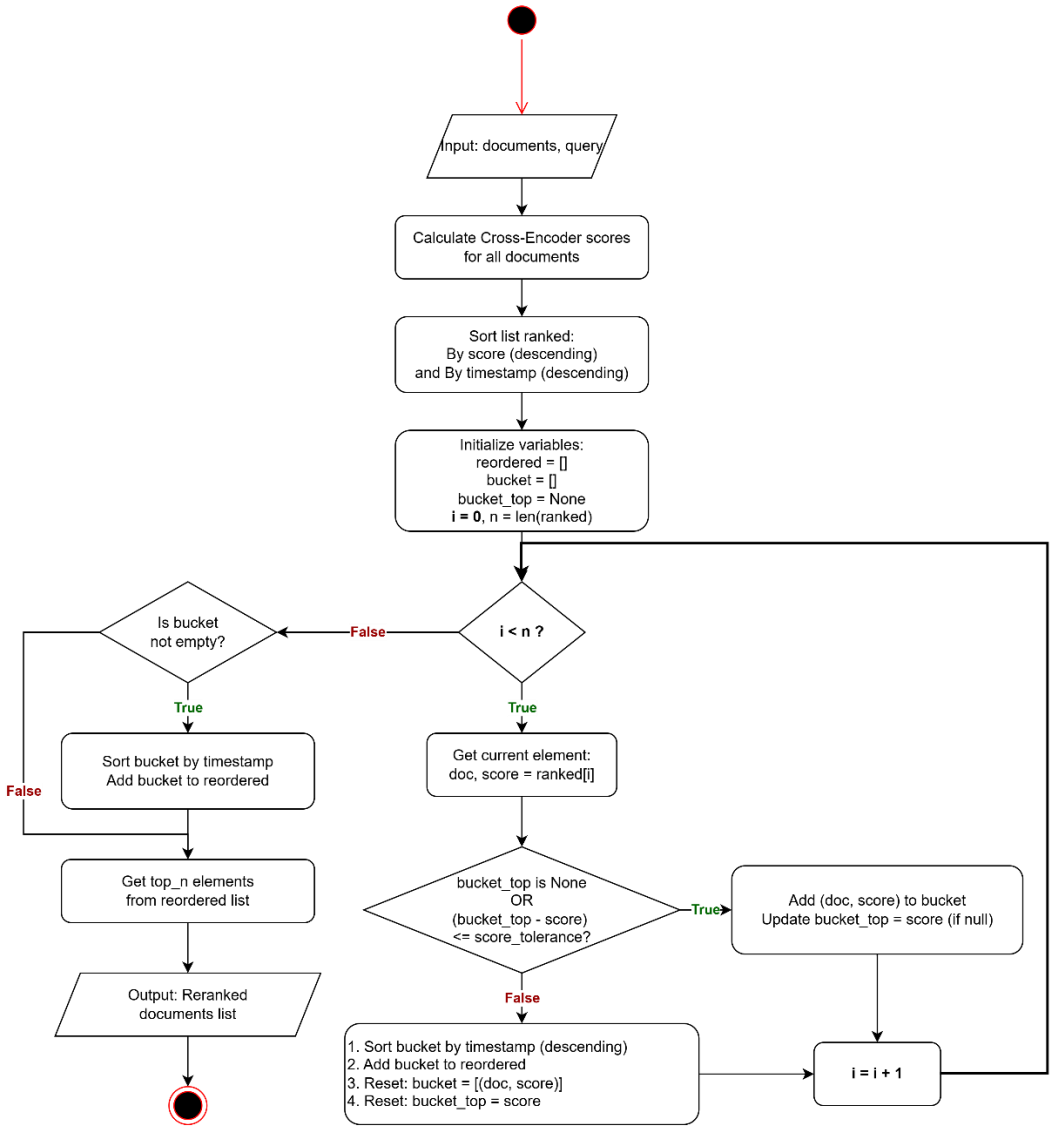

Figure 3.6. Soft tie break

Table 3.19. Important implementation parameters

Parameter	Current value	Design purpose
Parent chunk size / overlap	2,800 / 100 characters	Preserve policy context while controlling length
Child chunk size / overlap	400 / 50 characters	High-resolution semantic matching
Vector dimension	768	Match Vietnamese Bi-Encoder output
Vector distance	Cosine	Compare semantic direction
Qdrant candidate count	15	Broad first-stage recall
Reranker result count	6	Control prompt size and contamination
Reranker max length	512 tokens	Limit GPU memory consumption
Temporal tolerance	0.05	Prefer recent content only among near-equal scores

Parameter	Current value	Design purpose
Redis history window	5 messages	Provide context without excessive prompt growth

3.2.4. User interface design

The interface is implemented as a responsive single-page chat experience with modal authentication and administrator upload controls. The following reserved areas are intentionally empty so that final screenshots can be inserted after the deployment appearance has been approved.

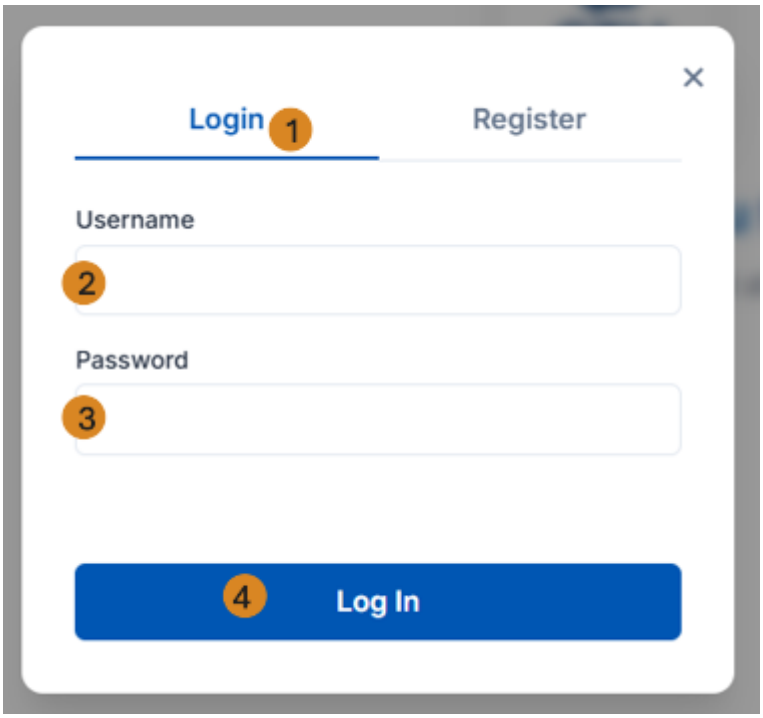

Figure 3.7. Login interface

Form Login include:

- 1. Login Page
- 2. Username for login
- 3. Password for login
- 4. Button Log in for submit login

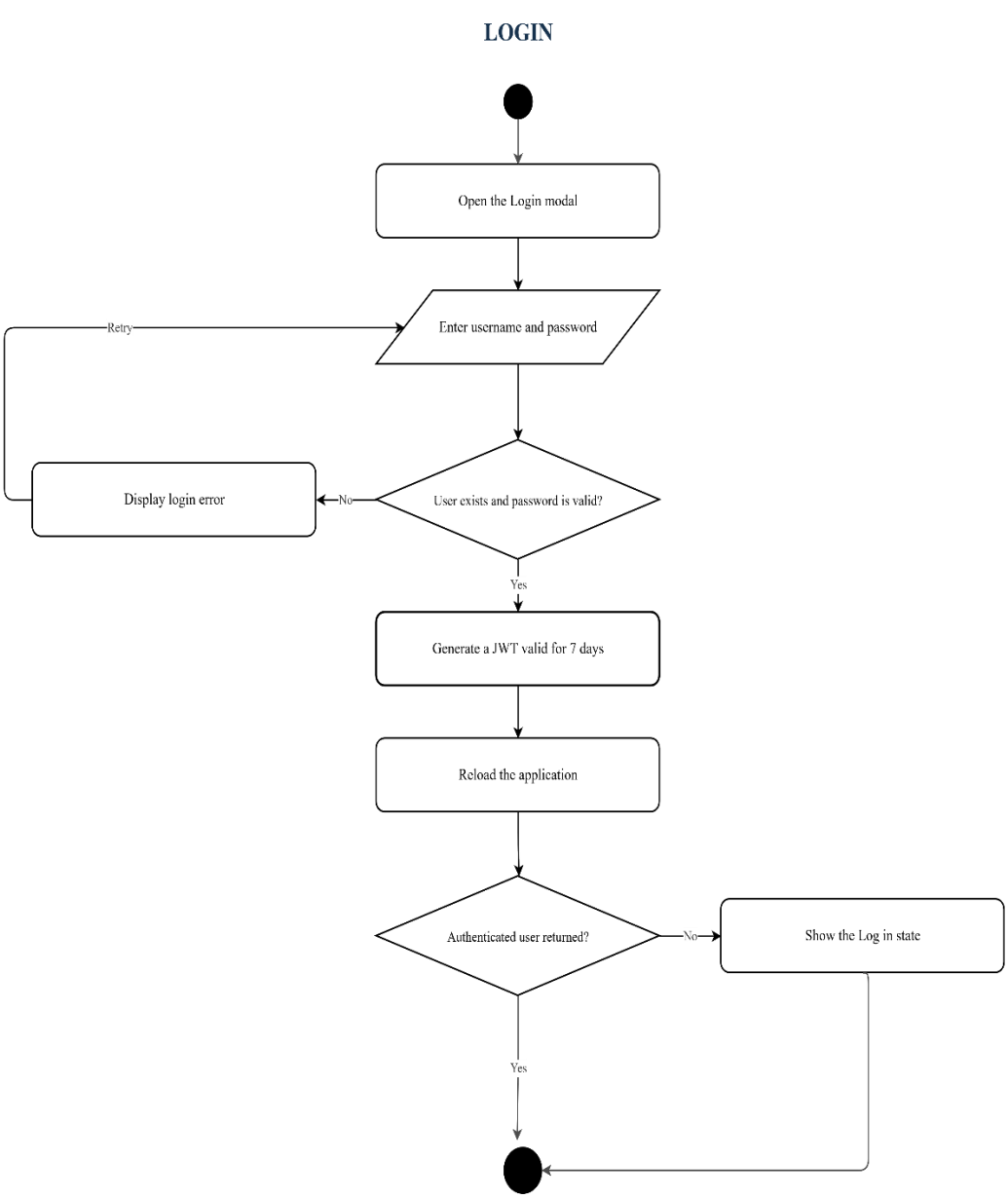

Figure 3.8. Login flowchart

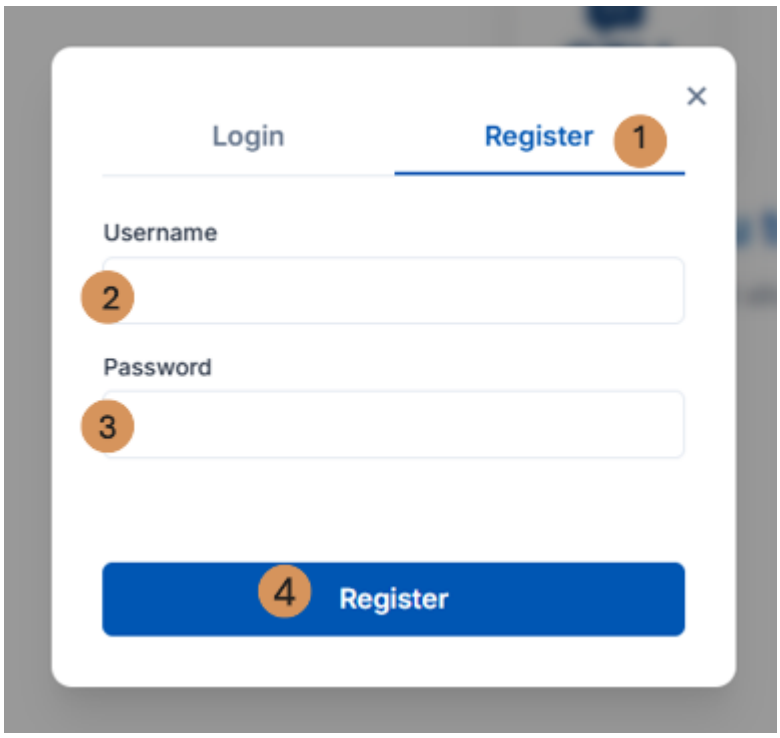

Figure 3.6. Registration interface

Register Form include:

- 1. Register page
- 2. Username for registration
- 3. Password for registration
- 4. Register button for submit registration

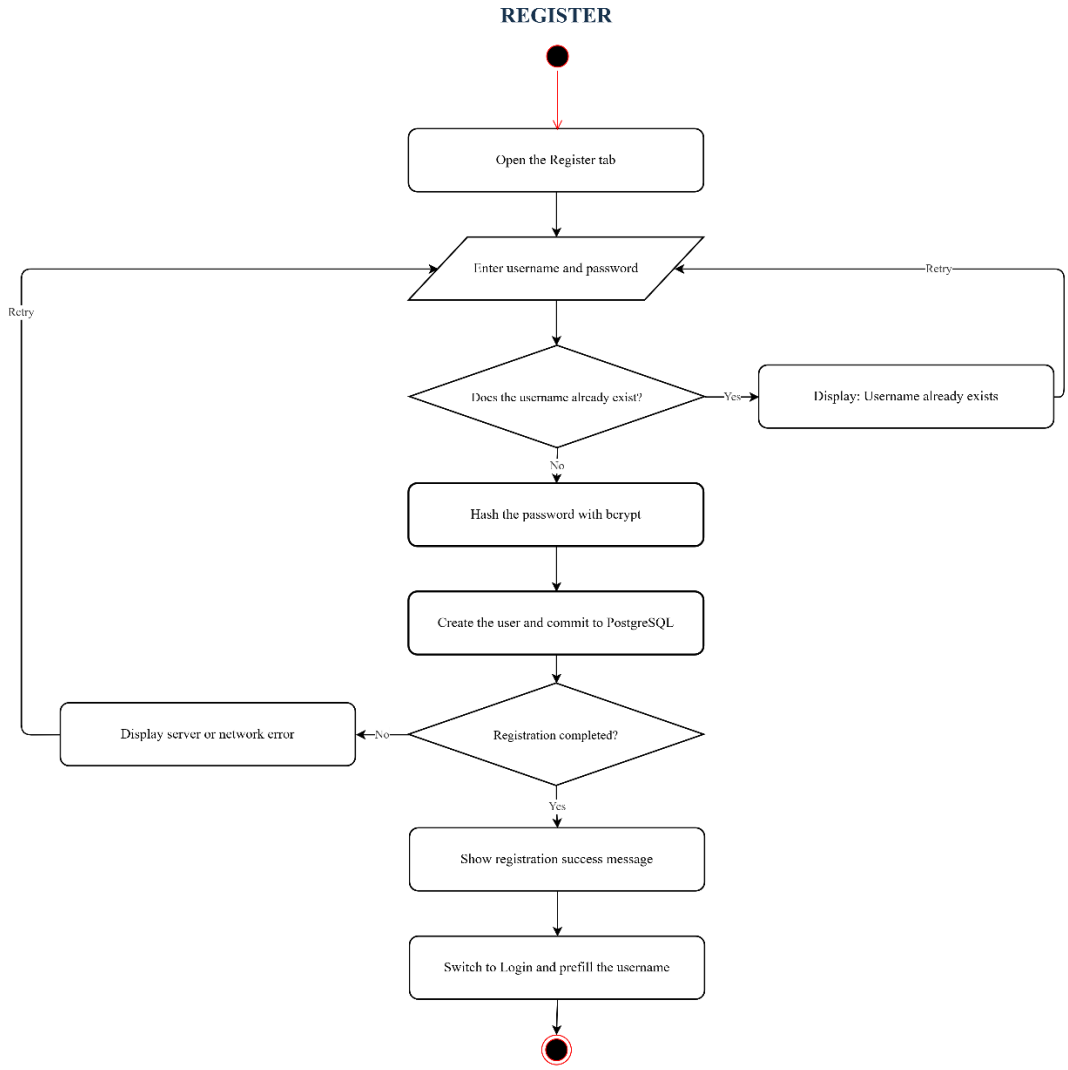

Figure 3.7. Registration flowchart

Figure 3.11. Main chatbot interface

Main Form include:

- 1. New Chat button for new session chat
- 2. History chat
- 3. Upload PDF (For admin)
- 4. Support (Information about project)
- 5. Logout (If use was login)
- 6. Area for user input message
- 7. Button to send message

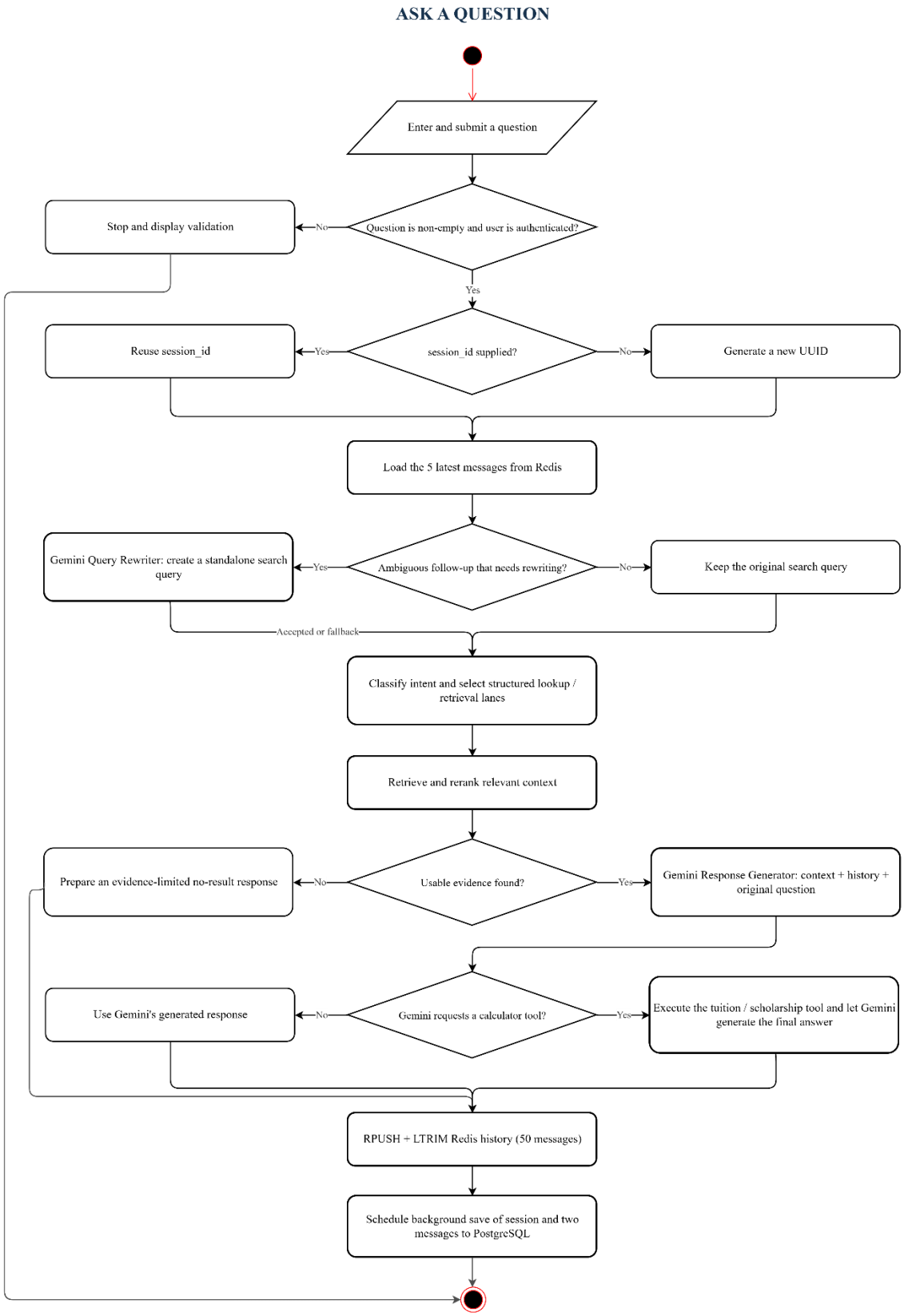

Figure 3.12. Ask a Question flow

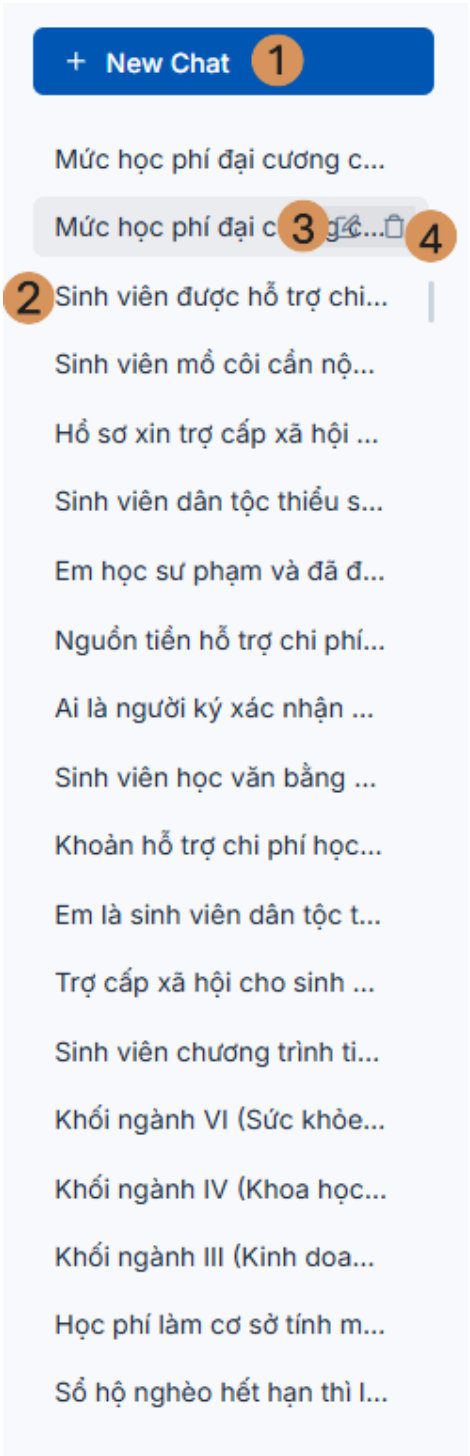

Figure 3.13. Conversation history and session controls

Session Management include:

- 1. New Chat button
- 2. History chat: Each cell is one history chat
- 3. Each history chat can be edit name
- 4. Each history chat can be delete

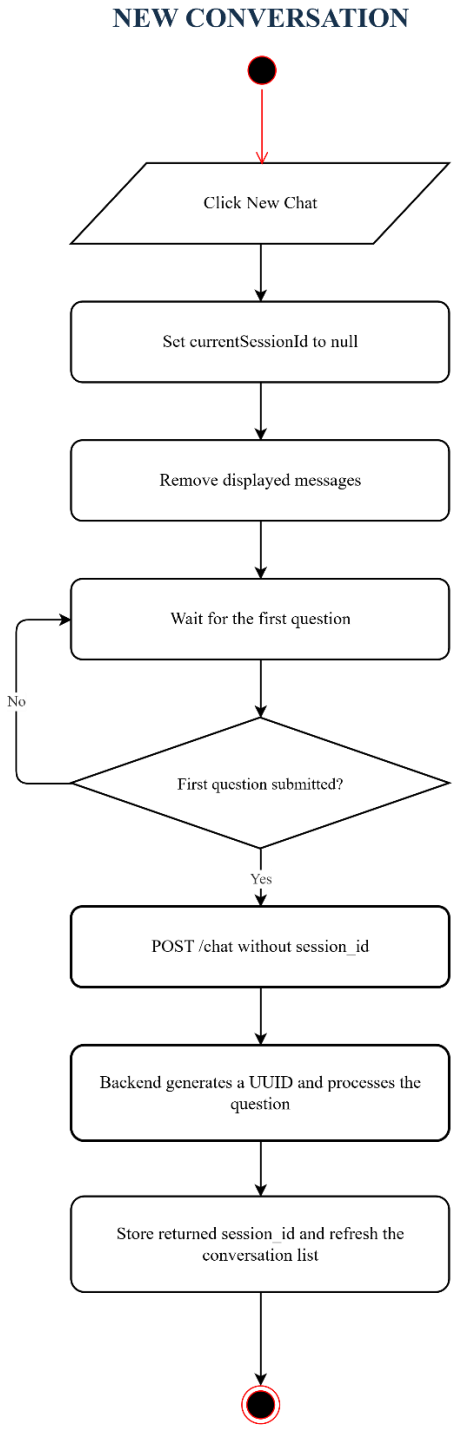

Figure 3.14. New Conversation flowchart

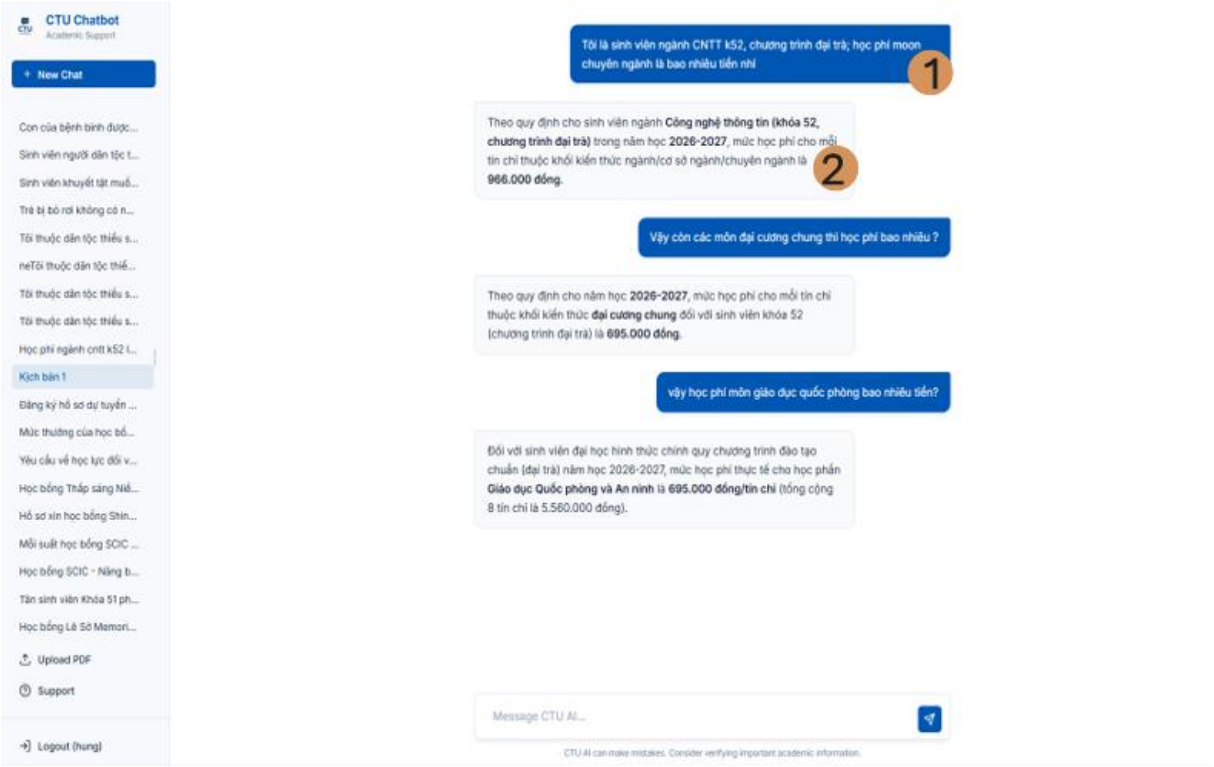

Figure 3.15. Chat answer presentation

In conversation:

- 1. Message form user
- 2. Message form AI

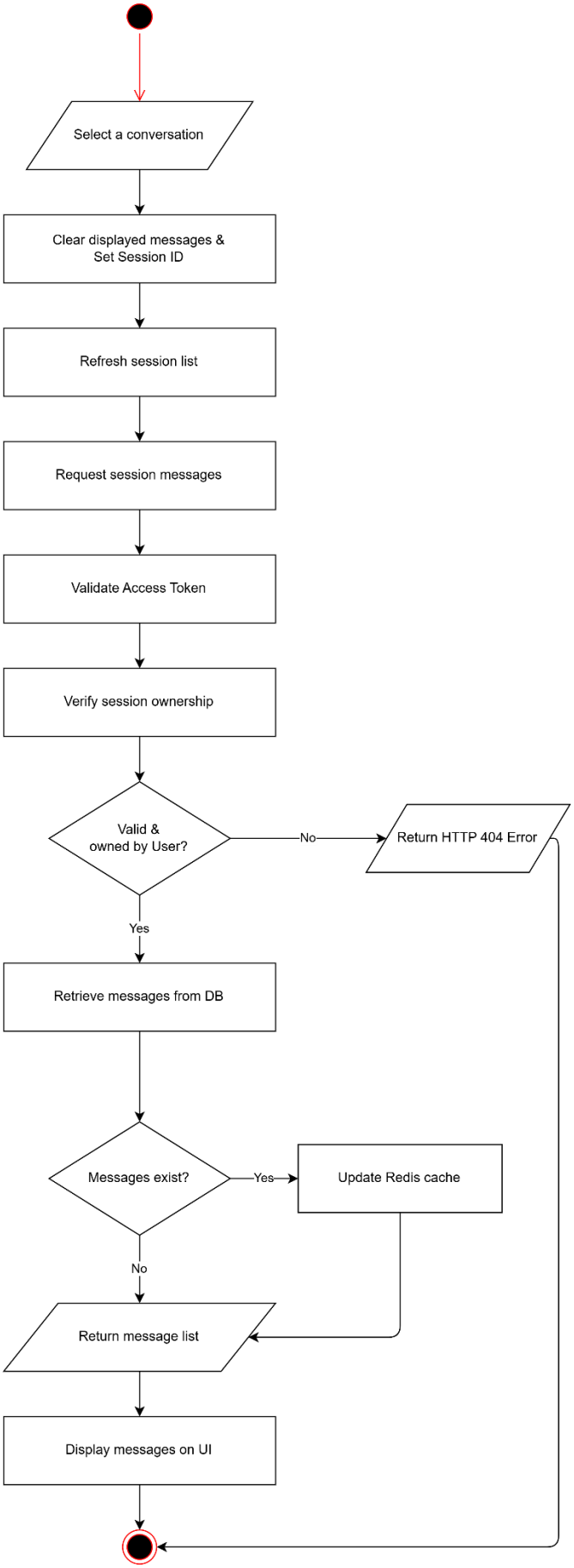

Figure 3.16. View Conversation Flow Chart

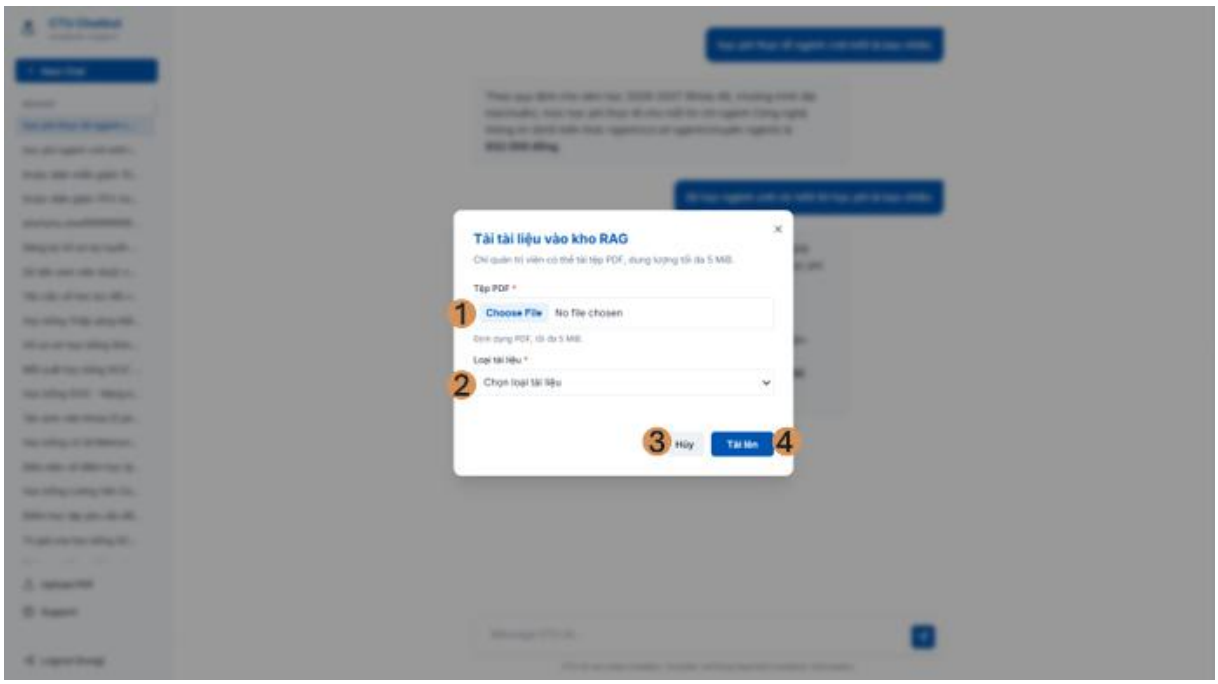

Figure 3.17. Administrator PDF upload interface

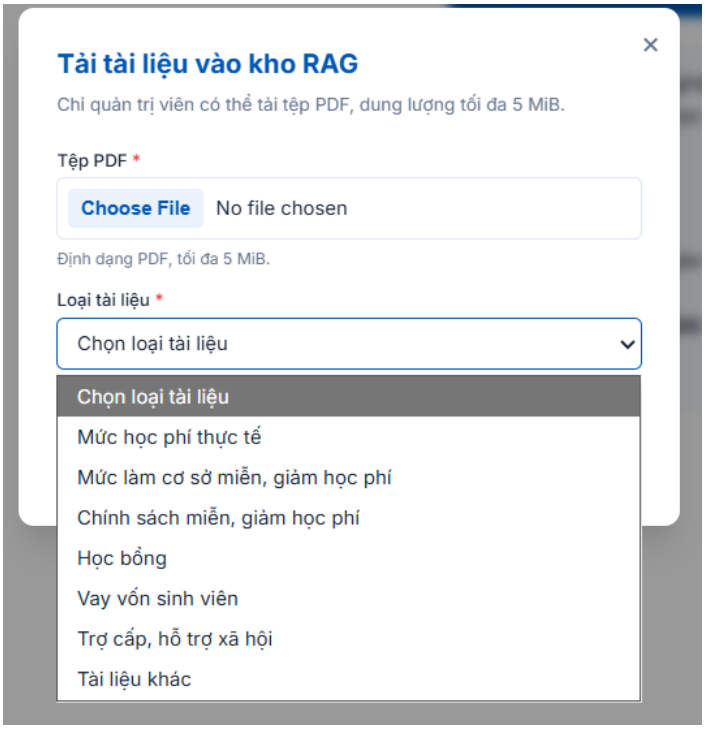

Figure 3.18. Document type selection list

Upload PDF Form include :

- 1. Choose File field
- 2. Select type of document
- 3. Cancel button for hide this form
- 4. Upload button for upload file

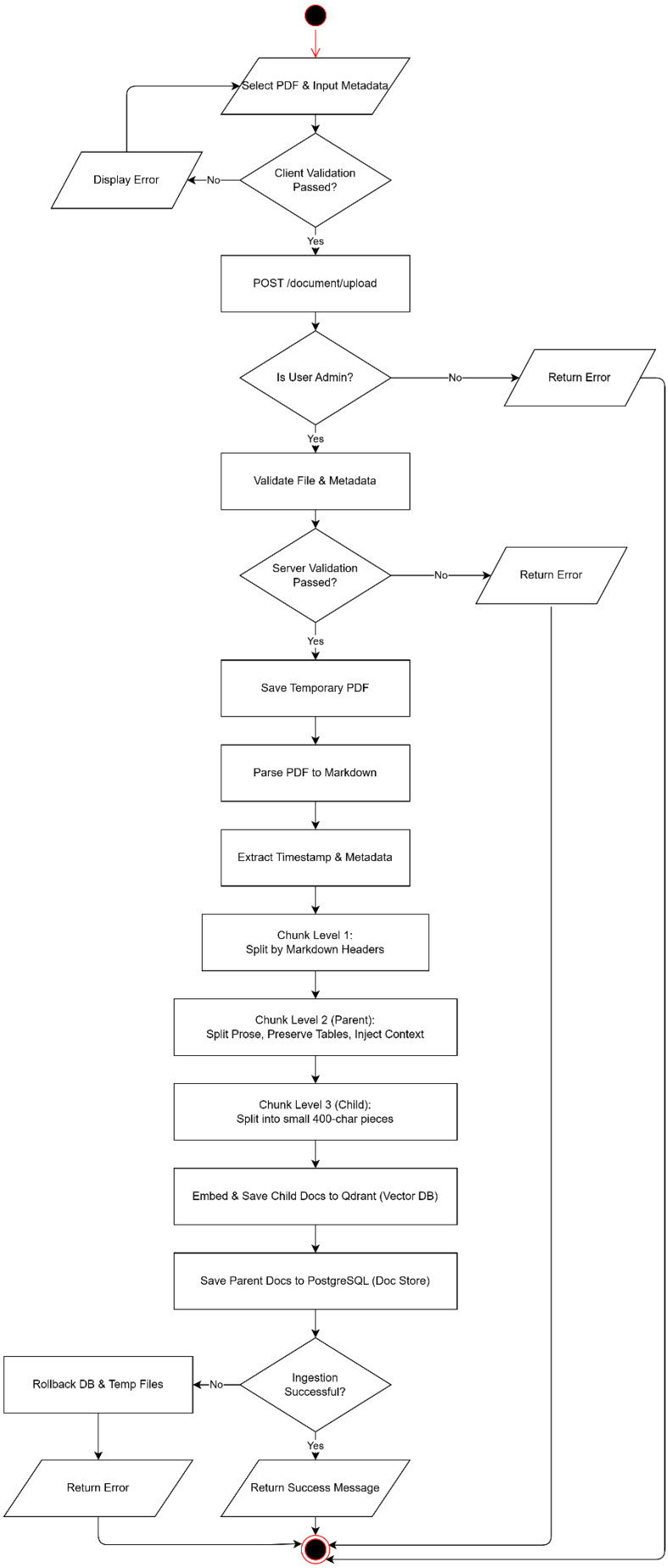

Figure 3.19. Upload PDF Flow Chart

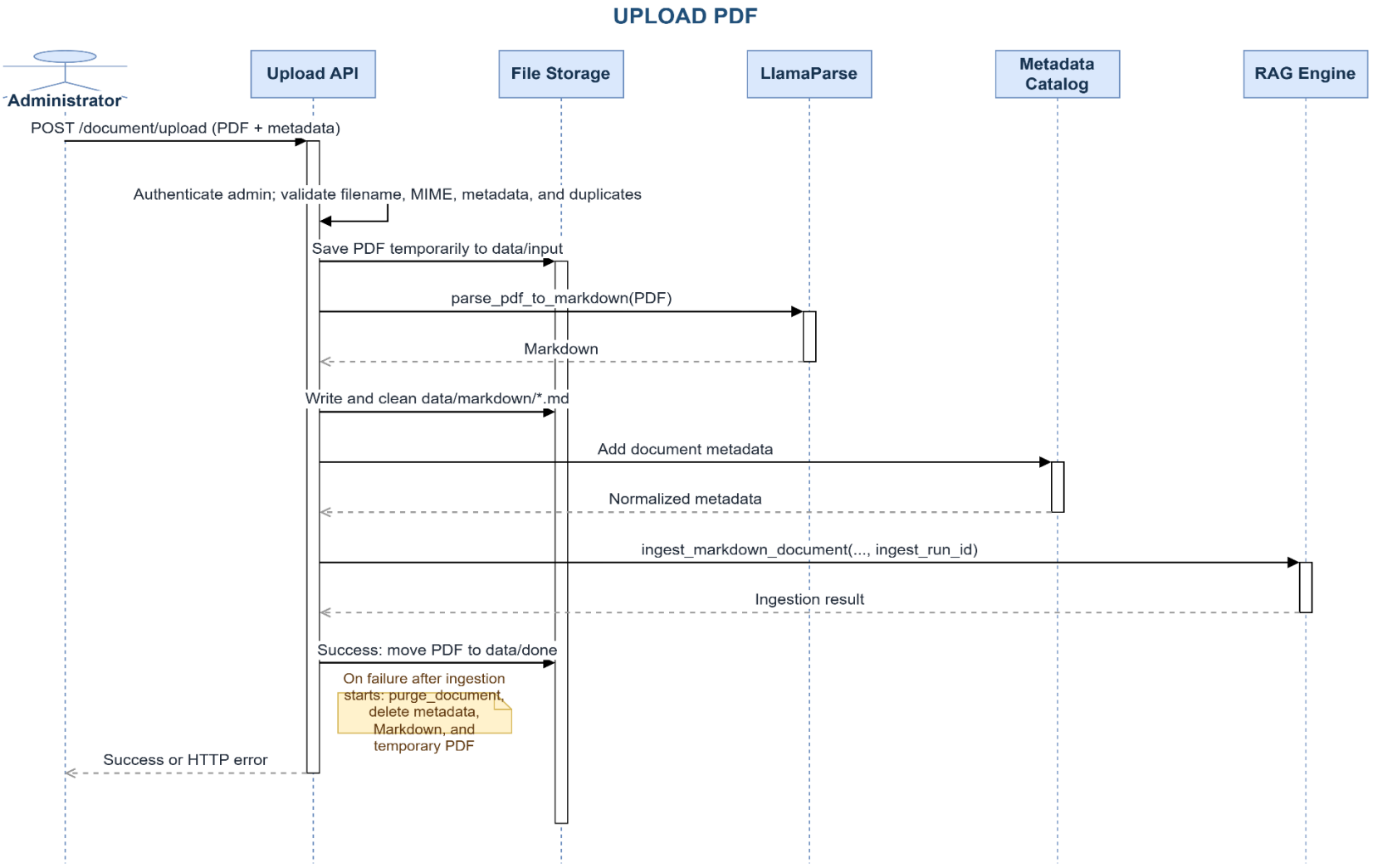

Figure 3.20. Upload PDF sequence diagram

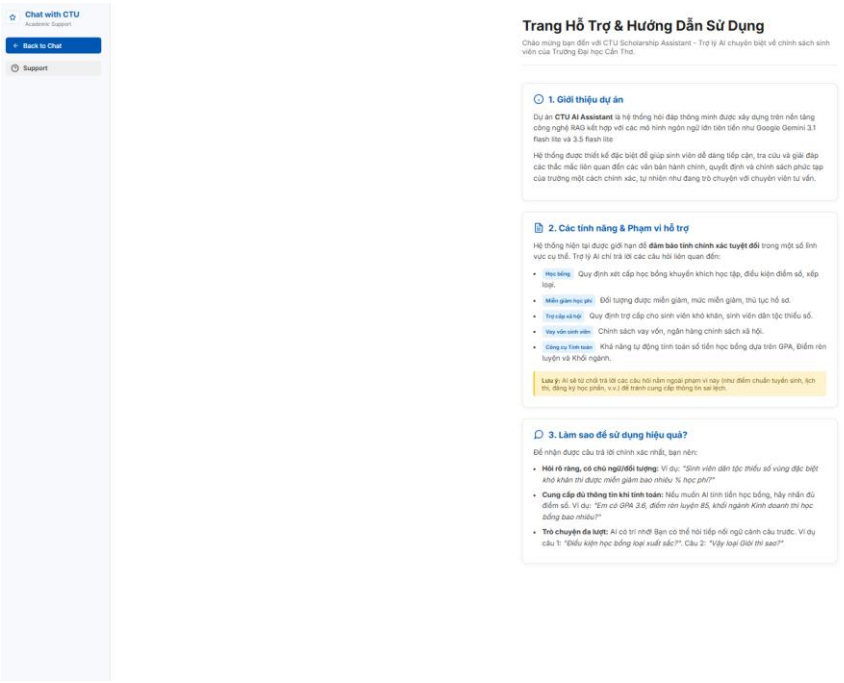

Figure 3.21. Support and policy-topic page

Support Form include:

- Information about Project Chatbot RAG CTU

3.3. TESTING

3.3.1. Test plan

Testing separates deterministic components from external model behavior. Unit tests verify validation, routing, catalog lookup, calculations, and scoring. Integration-style tests exercise the document upload transaction and authorization boundaries using controlled doubles for external systems. System evaluation uses the authenticated HTTP path so that cookies, request schemas, session behavior, and the real /chat orchestration are included.

Table 3.20. Testing strategy

Test level	Scope	Representative evidence	Pass criterion
Unit	Pure routing, lookup, calculation, and parser behavior	Intent, tuition catalog, finance tools, evaluator tests	Expected value or exception
Integration	API boundary plus coordinated components	Upload, metadata, authorization, rollback	HTTP status and complete state
System	Running browser-equivalent chat flow	Login cookie followed by POST /chat	Valid response and session

Test level	Scope	Representative evidence	Pass criterion
Security	Ownership, role, path, file, and token boundaries	Admin and upload validation cases	Unauthorized or unsafe input rejected

Table 3.21. Automated test inventory

Test file	Cases	Purpose
test_admin_authorization.py	2	Automated source test inventory
test_chat_dataset_eval.py	9	Automated source test inventory
test_document_metadata.py	15	Automated source test inventory
test_document_upload.py	13	Automated source test inventory
test_finance_tools.py	9	Automated source test inventory
test_query_intent.py	41	Automated source test inventory
test_tuition_catalog.py	15	Automated source test inventory
TOTAL	104	

3.3.1.1. Representative test cases

Table 3.22. Representative automated test cases

ID	Scenario	Expected result	Actual result	Status
TC-01	Student logs in with valid credentials	HTTP-only access cookie is issued	Cookie issued	Pass
TC-02	Non-admin attempts PDF upload	Request rejected with HTTP 403	HTTP 403	Pass
TC-03	Filename contains path traversal	Request rejected before writing	Rejected	Pass
TC-04	Question asks actual tuition while negating exemption	Route to actual tuition	Actual-tuition intent	Pass
TC-05	Course-name tuition lookup omits major	Find course record without false clarification	Record found	Pass
TC-06	Reduction percentage is outside 0-100	Calculation tool rejects input	Validation error	Pass
TC-07	Partial ingestion fails after indexing	Same-run vectors, parents, files, and metadata removed	Rollback completed	Pass
TC-08	Clear new question follows unrelated history	Original intent retained; no inherited entity	Rewrite skipped	Pass

3.3.1.2. Coverage and acceptance procedure

Automated execution should begin with deterministic unit tests because they isolate routing, normalization, calculation, and scoring defects without consuming external quotas. Integration tests then exercise authorization and transaction boundaries with controlled service doubles. A final live run is performed only after the application, corpus, model configuration, and database services are frozen. This sequence makes failures easier to localize and prevents a changing retrieval index from invalidating earlier results.

Authentication acceptance requires more than a successful login response. The test must retain the HTTP-only cookie, confirm that the current-user endpoint resolves the account, verify that protected endpoints reject an unauthenticated client, and demonstrate that one user cannot read or mutate another user's session. Administrator acceptance additionally verifies the persisted role at the upload endpoint; hiding the upload control in the browser is not considered authorization.

Retrieval acceptance uses representative questions from every policy lane. For each question, the reviewer records the classified intent, structured or vector path, applied metadata filters, retrieved sources, and final answer. Exact tuition cases must not fall back to a semantically related exemption-basis document. Scholarship, loan, and social-support cases must remain within their domains. Questions outside the indexed evidence must produce an abstention that does not invent a source or numeric value.

Ingestion acceptance verifies both the successful and interrupted paths. A successful upload must create classified Markdown, parent records, child vectors, metadata entries, and an active searchable version. Injected failure after each major step must leave no artifacts associated with the failed run and must preserve the previous active version. File validation is tested before external parsing so malformed, oversized, forged, or traversal filenames never become parser input.

The 100-question evaluation is the final system measurement, not a substitute for the preceding tests. It uses the same authenticated API flow as the browser and stores expected and actual answers for review. Before reporting accuracy, the evaluator configuration, timestamp, corpus checksum or index version, generator and rewriter model names, scoring version, and any failed infrastructure calls should be recorded.

Table 3.23. Manual acceptance evidence

Acceptance area	Required evidence	Failure interpretation
Authentication and ownership	Cookie flow, current user, cross-user denial	Security failure regardless of chat quality
Routing and catalog	Intent, lane, clarification, authoritative record	Wrong source class or monetary basis
Vector retrieval	Filters, child match, restored parent, reranker order	Recall, filtering, or ranking defect
Finance tools	Validated arguments, formula result, final explanation	Input, arithmetic, or verbalization defect
Ingestion	Complete success artifacts and clean rollback artifacts	Data-integrity or corpus-contamination risk
End-to-end evaluation	Authenticated request log and per-case evidence	Answer defect, abstention, or infrastructure failure

3.3.2. Results and analysis

3.3.2.1. Evaluation System

The evaluation dataset consists of 100 Vietnamese question–answer test cases, distributed equally across four domains: tuition exemption and learning-cost support, tuition, student loans, and scholarships. Each test case contains a question, an expected answer, and one or more expected source documents. The source information is retained for traceability but is not currently included in the automated scoring criteria.

Gemini 3.1 Flash-Lite is used as an LLM-as-a-judge to compare each actual chatbot response with its expected answer. The judge assigns a score from 0.0 to 1.0 and provides a pass/fail decision with a short explanation. A response passes only when its score is at least 0.55 and it correctly captures the essential facts in the expected answer. If the chatbot returns no answer or an API error occurs, the case receives a score of 0.0 and is marked as failed.

Accuracy = (Number of passed cases / Total number of cases) × 100%

Average score = (Σ judge score for all cases / Total number of cases) × 100%

Table 3.24. Chatbot accuracy evaluation results

Domain	Passed	Total	Accuracy	Average score
Tuition exemption and learning-cost support	25	25	100%	99.20%
Tuition	23	25	92%	94.00%
Student loans	24	25	96%	93.00%
Scholarships	23	25	92%	92.40%
Overall	95	100	95%	94.65%

The chatbot passed 95 out of 100 evaluation cases, achieving an overall accuracy of 95% and an average judge score of 94.65%. The strongest performance was observed in tuition exemption and learning-cost support, where all 25 cases passed. Student-loan questions achieved 96% accuracy, while tuition and scholarship questions each achieved 92%. These results indicate generally stable performance across all four evaluated domains, although several failures remain in tuition, student-loan, and scholarship questions.

The five failed cases mainly resulted from failures to obtain the required information or preserve important qualifying details. Cases 26, 60, 91, and 99 returned a no-result response even though the expected information was available in the reference data. These cases involved the K49 general tuition rate, VietinBank's age limit, the GPA requirement for the Lương Văn Can Scholarship, and the scholarship amount provided by the Tây Ninh Study Promotion Association. Case 42 correctly identified the 1.5 tuition multiplier but omitted the important exclusions for theses, projects, and graduation internships.

3.3.2.2. Evaluation Reranker Model

Table 3.25. Reranker evaluation results

Criterion	BGE Reranker v2-m3	GTE Multilingual Reranker Base
Shared evaluation set	10 questions	10 questions
Hit@6	70%	60%
Mean Reciprocal Rank	0.65	0.50
End-to-end accuracy	95%	94%
Main observation	Better ranking of fact-bearing parent passages	Faster, but lower retrieval quality

BGE Reranker v2-m3 produced the stronger retrieval only result, improving Hit@6 by ten percentage points and MRR by 0.15. The one-percentage-point end-to-end accuracy difference was considered supporting rather than primary evidence because final answers can also be affected by Gemini generation and LLM-as-a-judge variability. BGE was therefore selected as the most appropriate trade off for this dataset and hardware configuration.

3.3.2.3. Financial Function and Tool-Calling Evaluation

Table 3.26. Financial function and tool-calling results

Function	**Cases**	**Selection/path**	**Arguments**	**Result**	**Case pass**
Structured tuition lookup	10	10/10	10/10	10/10	10/10
Scholarship tool calling	10	10/10	9/10	10/10	9/10
Tuition-reduction tool calling	10	10/10	10/10	10/10	10/10
Overall	30	30/30	29/30	30/30	29/30

This experiment evaluates three financial execution paths used by the chatbot: structured tuition lookup, scholarship calculation, and tuition-reduction calculation. Thirty Vietnamese test cases were prepared, with ten cases for each function group. Each case was evaluated based on function/path selection, argument extraction, deterministic result correctness, and overall case success.

Twenty-nine of 30 cases passed all checks (96.67%). Tool/path selection and deterministic output checks both reached 100%. The single failure was Case 19: Gemini selected tinh_tien_hoc_bong but supplied an empty khoi_nganh value instead of the requested “chất lượng cao khóa 51”. The tool consequently returned the amount list for all sectors rather than highlighting the requested CTTT_K51 sector.

3.3.2.4. In Depth Scenarios Evaluation

Table 3.27 In Depth Scenarios Testcase

ID	Scenario	Test focus	Observed result	Status
TC-01	STEM student-loan policy	Long multi-turn context preservation	The chatbot maintained the same loan-policy context across questions about first-year eligibility, later-year eligibility, loan amount, interest, and collateral requirements.	Pass
TC-02	Teacher-training reimbursement policy	Conditional reasoning and reimbursement explanation	The chatbot maintained the reimbursement-policy context, explained mandatory service conditions, and provided the partial-reimbursement formula using F, T1, and T2.	Pass
TC-03	Missing-evidence follow-up	Grounding and abstention behavior	For both the initial unsupported rent-support question and the follow up about poor-household support, the chatbot stated that the information was not found and did not invent an amount or policy.	Pass
TC-04	Out-of-scope programming request	Scope adherence and refusal behavior	The chatbot correctly rejected repeated requests for Java code because they were outside the supported student finance domain and redirected the user to relevant policy topics.	Pass

The in depth scenarios were used to evaluate the chatbot's behavior in multi-turn conversations, especially its ability to preserve relevant context across follow-up questions and to avoid generating unsupported information. The tested scenarios showed that the chatbot could maintain the current policy context when users asked successive questions within the same topic and could continue answering based on previously established information. In addition, when a question referred to information that was not available in the indexed knowledge base, the chatbot correctly returned a no-result response instead of inventing a policy, amount, or eligibility condition. These results demonstrate that the system can handle conversational follow-ups while maintaining grounding and appropriate abstention behavior.

3.3.2.5. Soft Tie Break Evaluation

Table 3.28 Soft Tie Break Evaluation

Evaluation Criteria	**Plain Reranker**	**With Soft Tie-break**	**Improvement Assessment**
Accuracy (Hit Rate)	81 / 100	81 / 100 queries	Equal (Preserves 100% correct results, no document loss)
Average Rank	Rank 1.60	Rank 1.49	Soft Tie-break is better (Pulls correct documents closer to Top 1)
Number of Promoted Queries	0 queries	9 queries	Soft Tie-break is better (Pushes 9 queries from rank 3 to rank 2)
Number of Demoted Queries	0 queries	0 queries	Soft Tie-break is absolutely Safe (Does not decrease the rank of any query)

The Soft Tie break mechanism acts as a perfect fine filter when dealing with documents that have extremely high semantic similarity (original Reranker scores are almost identical). Instead of random ranking, the algorithm safely uses secondary metadata such as effective dates to intervene. As a result, it accurately demotes outdated or low-priority documents, making room to push the accurate documents to higher positions. This completely resolves the document version conflict issue (e.g., old vs. new regulations) without degrading the overall accuracy of the search system.

3.3.2.6. Heuristic Evaluation

Table 3.29 Heuristic Evaluation

Evaluation Criteria	**Metric (0 - 1)**	**Meaning & Measurement Method**
Context Precision	0.9954	Evaluates whether the retrieved context contains the answer at the top positions. Calculated based on the Mean Average Precision algorithm.
Answer Relevancy	0.9813	Assesses how closely the answer aligns with the question, without containing redundant information. Calculated using Cosine Similarity, normalized against the Ground Truth.
Context Recall	0.9605	Evaluates whether the retrieved context covers all the points of the standard Ground Truth. Calculated based on ROUGE-L Recall.
Faithfulness	0.9596	Measures the extent to which the generated answer is entirely based on the provided context no hallucination. Calculated via keyword overlap ratio (ROUGE-L Precision).
Answer Correctness	0.8698	The overall accuracy of the answer compared to the standard answer. It is a combination of keyword overlap (ROUGE Recall) and global semantic similarity.

The system demonstrates highly superior document search capabilities, evidenced by a Context Precision of 0.9954 and a Context Recall of 0.9605. For nearly 100% of the questions, it successfully finds the correct document containing the answer and pushes it to the Top 1 position, proving that the current chunking strategy and embedding model are highly suitable for the data domain. Building upon this outstanding retrieval, the bot generates highly relevant answers without fabricating external information, as confirmed by a Faithfulness score of 0.9596. Furthermore, an Answer Relevancy of 0.9813 highlights its ability to distill information and directly address the user's questions without rambling. By normalizing this relevancy against the Ground Truth, it is evident that the bot's answers are as sharp as human-provided samples. Finally, the system maintains stable content correctness with a score of 0.8698. This is actually an extremely high score for a strict measurement method that combines ROUGE and Cosine; it doesn't reach 0.99 mainly because the bot sometimes phrases things more smoothly and naturally rather than mechanically copy-pasting from the Ground Truth, slightly reducing the local vocabulary overlap ratio while completely preserving the overall semantic value.

4. CONCLUSION

4.1. PROJECT SUMMARY

The project delivers a complete web application for CTU student-finance policy support. It integrates secure account access, persistent conversations, structured tuition lookup, policy-domain routing, Vietnamese semantic retrieval, parent-child context recovery, multilingual reranking, grounded answer generation, controlled finance tools, and administrator document ingestion. The design distinguishes information retrieval from generation and distinguishes approximate semantic retrieval from exact monetary data.

The implementation addresses the central risk identified in Chapter 1: a fluent answer is not sufficient when several similar policy concepts contain different monetary values. Deterministic intent routing and business metadata separate actual tuition, exemption basis, exemption policy, scholarships, loans, and social support. Source

labels, no-result behavior, and tool validation make the response process more auditable than unconstrained prompting.

The software-engineering objectives are also represented in the architecture. PostgreSQL, Redis, and Qdrant have separate storage roles; APIs enforce authentication, ownership, and administration boundaries; upload validation protects the file boundary; and ingestion run identifiers support rollback. The automated tests cover routing, structured lookup, metadata integrity, finance tools, authorization, file validation, and evaluator behavior.

4.2. ACHIEVEMENT OF OBJECTIVES

Table 4.1. Achievement of project objectives

Objective	Implemented outcome	Status
Grounded policy answers	Labeled RAG context and explicit abstention instruction	Implemented
Vietnamese semantic retrieval	Vietnamese Bi-Encoder, parent-child retrieval, and BGE reranking	Implemented
Accurate tuition separation	Intent lanes, metadata filters, and structured tuition catalog	Implemented
Controlled calculations	Scholarship and payable-tuition tools with validation	Implemented
Conversation support	Redis recent context and PostgreSQL durable history	Implemented
Safe document growth	Administrator upload, validation, metadata, and rollback	Implemented

4.3. LIMITATIONS

The chatbot remains bounded by the active corpus. Missing, outdated, misclassified, or poorly parsed documents can cause abstention or retrieval errors. External Gemini, and LlamaParse services introduce availability, quota, and version dependencies. Local embeddings avoid an external embedding API but still require model download and compute resources. The reranker is substantially larger than the embedding model and is constrained by the available GPU and its configured 512-token input.

The current embedding selection is supported by Vietnamese and legal-retrieval evidence, but it has not yet been compared with newer multilingual alternatives on the exact CTU corpus. The LLM-as-a-judge evaluator provides a consistent scoring rubric,

but its judgments may vary slightly across runs because the evaluation itself is model-based. Therefore, the reported results should be interpreted together with the fixed dataset, evaluation prompt, model version, and execution configuration.

4.4. FUTURE WORK

Table 4.2. Future work

Improvement	Expected benefit
Run a controlled embedding benchmark and hybrid BM25-dense retrieval experiment	Evidence-based model and retrieval selection
Display source citations and policy-effective dates in answers	Improved auditability and user trust
Add administration for document status, version, and rollback visibility	Safer corpus operations
Using UAT with students and staff	Usability evidence
Evaluate multilingual and mobile-client support	Broader access

The highest-priority next step is not indiscriminate model replacement. The system should first freeze a representative corpus, construct relevance judgments, and measure candidate retrieval and end-to-end behavior under identical conditions. This approach preserves the evidence first principle used throughout the project and ensures that added complexity is justified by measurable improvement.

REFERENCES

[1] BKAI-HUST Foundation Models Lab, "bkai-foundation-models/vietnamese-bi-encoder," Hugging Face model card.https://huggingface.co/bkai-foundation-models/vietnamese-bi-encoder. [Accessed: Jul. 23, 2026].

[2] N. Q. Duc, L. H. Son, N. D. Nhan, N. D. N. Minh, L. T. Huong, and D. V. Sang, "Towards Comprehensive Vietnamese Retrieval-Augmented Generation and Large Language Models," arXiv:2403.01616, 2024.

[3] Beijing Academy of Artificial Intelligence, "BAAI/bge-reranker-v2-m3," Hugging Face model card. [Online]. Available: https://huggingface.co/BAAI/bge-reranker-v2-m3. [Accessed: Jul. 23, 2026].

[4] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation," arXiv:2402.03216, 2024.

[5] C. Li, Z. Liu, S. Xiao, and Y. Shao, "Making Large Language Models a Better Foundation for Dense Retrieval," arXiv:2312.15503, 2023.

[6] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in Proc. EMNLP-IJCNLP, 2019, pp. 3982-3992.

[7] Y. A. Malkov and D. A. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," IEEE TPAMI, vol. 42, no. 4, pp. 824-836, 2020.

[8] Qdrant, "Qdrant Documentation." [Online]. Available: https://qdrant.tech/documentation/. [Accessed: Jul. 23, 2026].

[9] PostgreSQL Global Development Group, "PostgreSQL 15 Documentation." [Online]. Available: https://www.postgresql.org/docs/15/. [Accessed: Jul. 23, 2026].

[10] Redis, "Redis Documentation." [Online]. Available: https://redis.io/docs/latest/. [Accessed: Jul. 23, 2026].

[11] LangChain, "LangChain Documentation." [Online]. Available: https://python.langchain.com/docs/. [Accessed: Jul. 23, 2026].

[12] LlamaIndex, "LlamaParse Documentation." [Online]. Available: https://developers.llamaindex.ai/python/cloud/llamaparse/. [Accessed: Jul. 23, 2026].

APPENDICES

Table A.1. Public API endpoint summary

Method	Path	Authorization	Purpose
POST	/auth/register	Public	Create a student account
POST	/auth/login	Public	Issue HTTP-only access cookie
POST	/auth/logout	Public	Delete access cookie
GET	/auth/me	User	Return current identity and role
POST	/chat	User	Answer a question in a new or existing session
GET	/sessions	User	List owned sessions
GET	/sessions/{id}/messages	Owner	Load messages and repopulate recent history
PUT	/sessions/{id}	Owner	Rename a session
DELETE	/sessions/{id}	Owner	Delete a session and Redis history
POST	/document/upload	Admin	Validate, parse, classify, and ingest a PDF

Dataset use for evaluation:

- https://drive.google.com/file/d/1t3cWGD4KV4mfZYsU1l6pdRJvJzcEkY-b/view?usp=sharing

Result of evaluation:

- https://drive.google.com/file/d/1VvmoDJekPtvGMYTq7rQXB8vcg2Ypeo-z/view?usp=sharing

Dataset use for test tool calling:

- https://drive.google.com/file/d/1vzmh72ogQyU3xO3RgnWlqt1gjCR6nGzl/view?usp=sharing

Scenarios TC-01->04

- https://docs.google.com/spreadsheets/d/1gjsxvTI8JxRbpjlJ8YpL7R0GSjAdGh5x/edit?usp=sharing&ouid=117435450070938810408&rtpof=true&sd=true
- https://docs.google.com/spreadsheets/d/1hCuc_uNjgWNAxAYkW0LmbuUwt74Hde-u/edit?usp=sharing&ouid=117435450070938810408&rtpof=true&sd=true

- https://docs.google.com/spreadsheets/d/1FINZP2q1IJ1bd6luBeBmwYe9kAoAp7uO/edit?usp=sharing&ouid=117435450070938810408&rtpof=true&sd=true
- https://docs.google.com/spreadsheets/d/1aiPqt3A1kQm6xoEwgf0CuUHdJ6Ecr03w/edit?usp=sharing&ouid=117435450070938810408&rtpof=true&sd=true